ФАХОВИЙ КОЛЕДЖ

Теорія ймовірностей та математична статистика

Зміст

1 Практичне заняття No2. Визначення ймовірності подій **5**
1.1 Мета: . . . 5
1.2 Завдання: . . . 5
1.3 Розв’язання . . . 5
1.4 Таблиця застосованих механізмів валідації . . . 7
1.5 Висновок: . . . 7
1.6 Правки: . . . 7

2 Практичне заняття №4 **8**
2.1 Тема 3. . . . 8
2.2 Мета: . . . 8
2.3 Математична модель обчислення ймовірності . . . 8
2.4 Формулювання задачі. . . . 8
2.4.1 Приклад задачі що рішає модель. . . . 8
2.5 Математична модель вибірки. . . . 9
2.6 Перетворення формули ймовірності. . . . 11
2.7 Декомпозиція обчислення. . . . 11
2.8 Ефективне обчислення біноміальних коефіцієнтів. . . . 12
2.9 Остаточна формула розподілу. . . . 13
2.9.1 Таблиця функцій алгоритму моделі . . . 15
2.10 Обчислення ймовірності події. . . . 20
2.10.1 Допоміжні функції . . . 20

3 Тема 4. Bernoulli **23**
3.1 Мета . . . 23
3.2 Теорія . . . 23
3.3 Задача . . . 23
3.4 Функція вибору стратегії обчислення . . . 25

4 Практичне заняття №5. ЗМІ. Тема 4. Laplace, Poisson **27**
4.1 Мета . . . 27

4.2 Теорія . . . 27
4.3 Локальна теорема Лапласа . . . 27
4.4 Інтегральна теорема Лапласа . . . 28
4.5 Пуасона . . . 30
4.6 Загальна порівняльна характеристика . . . 31
4.7 Завдання 47 . . . 31
4.7.1 Таблична функція Лапласа . . . 32
4.7.2 Щільність нормального розподілу . . . 32
4.7.3 UML diagrams: Bernoulli, Laplace, Poisson . . . 33
4.8 Висновок . . . 34

5 Практичне заняття №7. ЗМІ. Тема 4 35
5.1 Числові характеристики дискретної випадкової величини . . . 35
5.2 Математичне сподівання . . . 36
5.3 Квадрат очікування . . . 37
5.4 Дисперсія . . . 37
5.5 Середнє квадратичне відхилення . . . 38
5.6 Результат виконання . . . 38

6 Випадкові величини і процеси. Дискретні випадкові величини 39
6.1 Завдання 6.1 . . . 39
6.2 Завдання 6.7 . . . 41
6.3 Висновок . . . 43

7 Випадкові величини і процеси. Неперервні випадкові величини 43
7.1 Завдання 8 . . . 44
7.2 Клас Statistics . . . 45
7.2.1 Математичне сподівання . . . 46
7.2.2 Математичне сподівання квадрата . . . 46
7.2.3 Дисперсія . . . 47
7.2.4 Середнє квадратичне відхилення . . . 47
7.3 Додаток . . . 49
7.3.1 Вибіркове середнє . . . 49
7.3.2 Математичне сподівання квадрата . . . 50

7.3.3 Дисперсія . . . 50
7.3.4 Виправлена дисперсія (Незміщена оцінка дисперсії.) . . . 50
7.3.5 Середнє квадратичне відхилення . . . 51
7.3.6 Виправлене середнє квадратичне відхилення . . . 51
7.4 Виправлення . . . 51

8 Щільність стандартного нормального розподілу 52

9 Емпірична функція розподілу 53

10 ANOVA. 53
10.1 Мета: . . . 54
10.2 Постановка задачі . . . 54
10.3 Гіпотези . . . 54
10.4 Інтерпретація . . . 55
10.5 Підготовка даних . . . 55
10.6 Формування вибірок . . . 56
10.7 Розрахунок ANOVA . . . 56
10.8 Критерій Фішера . . . 58
10.9 Критичне значення через квантиль F розподілу . . . 58
10.10 Висновок . . . 59
10.10.1. Перевірка гіпотези . . . 60
10.10.2. Стійкість по роках . . . 61
10.11 Реалізація: . . . 61

11 Тема 6. Інтервальні оцінки параметрів 67
11.1 Мета . . . 67
11.2 Завдання . . . 67
11.3 Розв’язання для Задачі №1: . . . 68

12 Реалізація обчислення 70
12.1 Аналіз . . . 72
12.1.1 Закономірності по завданнях . . . 74
12.2 Висновок . . . 75

1 Практичне заняття No2. Визначення ймовірності подій

1.1 Мета:

- Навчитися розв'язувати задачі з теорії ймовірностей у програмному коді `C++`. Ознайомитися із застосуванням шаблонних функцій та механізму обмеження типів `concepts` на прикладі обчислення ймовірнісних характеристик.

1.2 Завдання:

- При стрільбі із гвинтівки відносна частота влучення в мішень дорівнює 0,85. Скільки буде влучень, якщо проведено 120 пострілів? [17]

1.3 Розв'язання

- Функція обчислення повинна реалізувати математичне сподівання кількості влучень у схемі Бернуллі. Якщо ймовірність влучення при одному пострілі дорівнює p, а кількість пострілів n, тоді очікувана кількість влучень:

$$E(X) = n \cdot p$$

- Для перевірки коректності введених даних використаємо шаблонну функцію з обмеженнями типів `concepts`.

```
template<std::integral ShotType, std::floating_point ProbType>
bool validate_input(ShotType shots, ProbType probability)
```

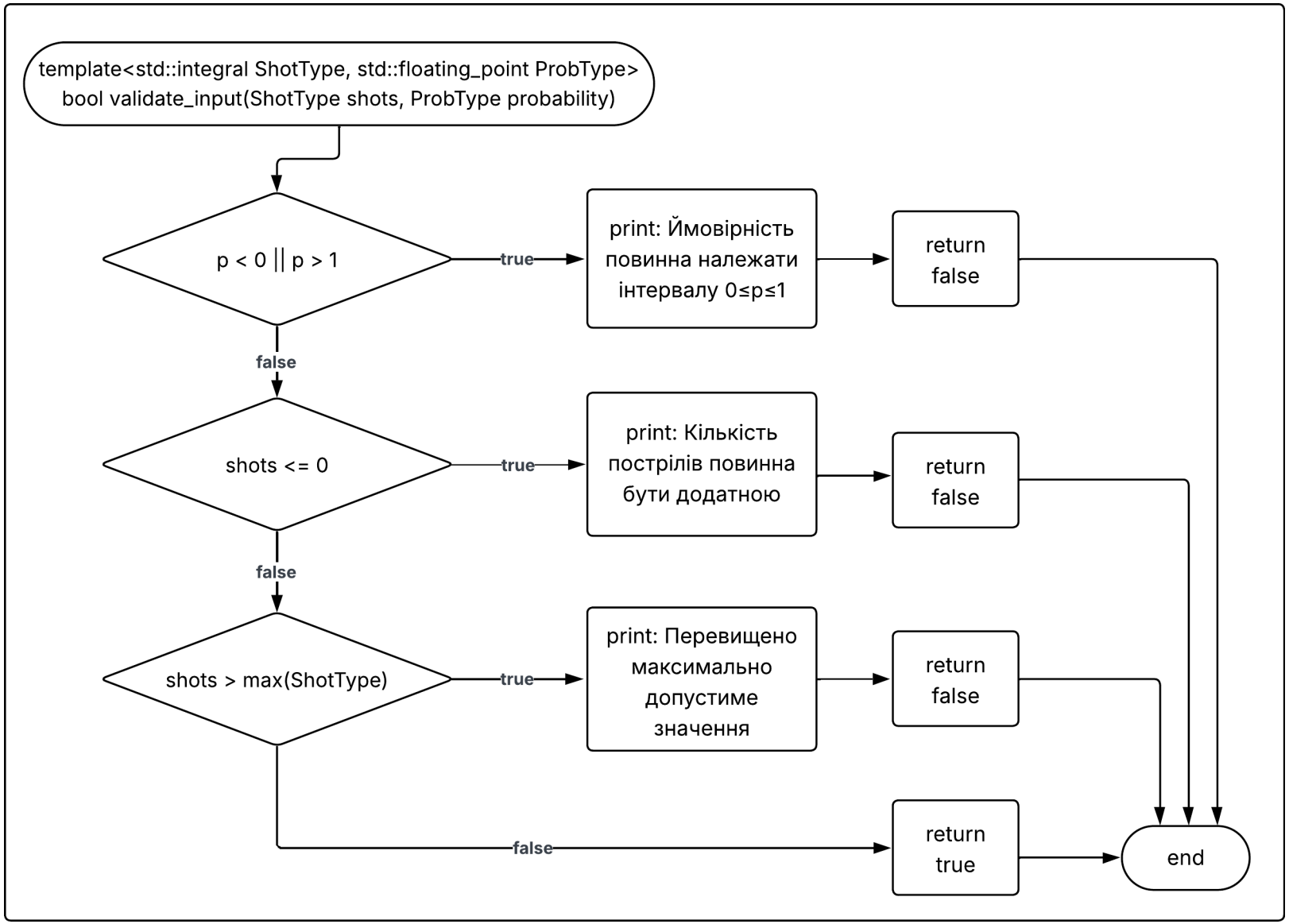

Рис. 1 Діаграма перевірок

1.4 Таблиця застосованих механізмів валідації

Елемент	Де використано	Призначення
`template<...>`	`validate_input`	Дозволяє створити узагальнену функцію для різних числових типів
`ShotType`	`validate_input`	Тип для кількості пострілів
`ProbType`	`validate_input`	Тип для ймовірності
`std::integral`	`template<std::integral ShotType>`	Обмежує параметр лише цілими типами
`std::floating_point`	`template<std::floating_point ProbType>`	Обмежує параметр лише типами з плаваючою комою
`std::numeric_limits<T>`	`validate_input`	Надає межі числових типів
`std::numeric_limits<T>::max()`	`validate_input`	Повертає максимальне значення для типу `T`

1.5 Висновок:

Опрацьовано використання шаблонної функції як механізму перевірки коректності введених даних перед виконанням основного алгоритму обчислення. Для цього застосовано параметри типів шаблону ShotType та ProbType, семантичні назви яких відображають їх призначення у програмі. Параметр ShotType використовується для представлення типу кількості пострілів (shot означає постріл), а ProbType для представлення типу ймовірності (prob є скороченням від probability). Обмеження типів реалізовано за допомогою механізму concepts, що дозволяє гарантувати використання цілого типу для кількості пострілів та типу з плаваючою комою для значення ймовірності. Додатково використано засіб стандартної бібліотеки std::numeric_limits для контролю допустимого діапазону значень цілого типу та запобігання переповненню.

1.6 Правки:

Причина	Дата	Що відкореговано
TIMC:	2026-02-10	Відкориговано вивід кількості пострілів. Дано округлення std::round(result).

2 Практичне заняття №4

2.1 Тема 3.

2.2 Мета:

Опрацювання основних теоретичних положень теорії ймовірностей, правил додавання і множення ймовірностей, умовної ймовірності та властивостей залежних і незалежних подій. У роботі також розглядається формалізація типових імовірнісних задач та побудова узагальненої обчислювальної моделі на основі задачі 29, в якої побудовано узагальнену математичну модель обчислення ймовірності для вибірки з (N) об'єктів, поділених на k типів, що дозволяє задавати різні параметри експерименту без прив'язки до конкретної задачі. Модель реалізує декомпозицію формули розподілу, містить перевірки коректності параметрів та використовує оптимізоване обчислення біноміальних коефіцієнтів без прямого використання факторіалів.

2.3 Математична модель обчислення ймовірності

2.4 Формулювання задачі.

2.4.1 Приклад задачі що рішає модель.

Серед 12 телевізорів, які надійшли у ремонт, у п'яти з них треба замінити електронні компоненти, а у інших замінити деталі корпусу. Яка ймовірність того, що серед чотирьох телевізорів, взятих навмання майстром для ремонту, хоча б двом треба замінити електронні компоненти.

Для розв'язання задачі спочатку будуємо математичну модель вибірки без повернення з множини об'єктів, поділених на типів. Модель дозволяє обчислювати ймовірність конкретної конфігурації кількостей об'єктів різних

типів у вибірці. Після цього ймовірність події задачі визначається як сума ймовірностей всіх конфігурацій, що задовольняють умову події.

2.5 Математична модель вибірки.

Нехай N об'єктів (12 телевізорів), розбитих на k типів (`2 типи в задачі{5 електр.комп.; 7 корпус}`)

Кількість об'єктів кожного типу:

$$N_1, N_2, ..., N_k$$

де

$$N_i \geq 0$$

$$i = 1, ..., k$$

Виконується умова узгодженості кількостей

$$N = N_1 + N_2, \cdots + N_k$$

або

$$N = \sum_{i=1}^{k} N_i$$

Нехай серед вибраних m об'єктів виявилося $x_1, x_2, ..., x_k$ об'єктів відповідно $1, 2, ..., k$-го типу.

Тоді $0 \leq x_i \leq Ni$ де $i = 1, ..., k$

та

$$m = x_1 + x_2 + ... + x_k$$

або

$$\sum_{i=1}^{k} x_i = m$$

Ймовірність того, що у вибірці з m об'єктів буде рівно $x_1, x_2, ..., x_k$ об'єктів відповідних типів, задається розподілом:

$$P(x_1, x_2, ..., x_k) = \frac{\binom{N_1}{x_1}\binom{N_2}{x_2}...\binom{N_k}{x_k}}{\binom{N}{m}}$$

де

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Це комбінація, без повторень, порядок слідування не враховується.

2.6 Перетворення формули ймовірності.

Підставляємо означення біноміального коефіцієнта:

$$P(x_1, x_2, \dots, x_k) = \frac{\frac{N_1!}{x_1!(N_1-x_1)!} \cdot \frac{N_2!}{x_2!(N_2-x_2)!} \cdots \frac{N_k!}{x_k!(N_k-x_k)!}}{\frac{N!}{m!(N-m)!}} = \frac{\prod_{i=1}^{k} \frac{N_i!}{c_i!(N_i-c_i)!}}{\frac{N!}{m!(N-m)!}}$$

Перевертаємо знаменник:

$$P(x_1, \dots, x_k) = \frac{m!(N-m)!}{N!} \cdot \prod_{i=1}^{k} \frac{N_i!}{x_i!(N_i-x_i)!}$$

2.7 Декомпозиція обчислення.

Глобальний множник:

$$A(N, m) = \frac{m!(N-m)!}{N!}$$

Добуток по типах

$$B(N_i, x_i) = \frac{N_i!}{x_i!(N_i-x_i)!}$$

Тепер формула має вигляд:

$$P(x_1, \dots, x_k) = A(N, m) \cdot \prod_{i=1}^{k} B(N_i, x_i)$$

2.8 Ефективне обчислення біноміальних коефіцієнтів.

Так як біноміальний коефіцієнт має симетрію:

$$\binom{n}{r} = \binom{n}{n-r}$$

Ми можемо обрати найменший параметр для обчислення:

$$r' = \begin{cases} r, & r \le n-r \\ n-r, & r > n-r \end{cases}$$

Тоді

$$r' = min(r, n-r)$$

маємо добуток з найменшою кількістю кроків.

Замінимо факторіальну реалізацію біноміальної функції `B` на еквівалентне обчислення через послідовний добуток дробів, бо факторіали дуже швидко ростуть:

$$\binom{n}{r} = \frac{n(n-1)(n-2)\cdots(n-r+1)}{r!}$$

$$\binom{n}{r} = \prod_{t=1}^{r'} \frac{n - r' + t}{t}$$

Наприклад якщо $n = 100, r = 97$ то рахувати через 97 множень неэффективно, але $n - r = 3$, $r' = min(97, 3) = 3$

$$\binom{100}{97} = \prod_{t=1}^{3} \frac{100 - 3 + t}{t} = \frac{98}{1} \cdot \frac{99}{2} \cdot \frac{100}{3}$$

$\binom{100}{97}$ теж саме що $\binom{100}{3}$, але кроків лише 3.

2.9 Остаточна формула розподілу.

Повертаємося до формули ймовірності і замінюємо функції $A(N, m)$, $B(N_i, x_i)$ добутком.

Оскільки:

$$\binom{N}{m} = \frac{N!}{m!(N-m)!}$$

Ми можемо обернути дріб:

$$\frac{1}{\binom{N}{m}} = \frac{1}{\frac{N!}{m!(N-m)!}}$$

Отримаємо:

$$A(N,m) = \frac{m!(N-m)!}{N!} = \frac{1}{\binom{N}{m}}$$

Покажемо `B` як $\binom{N_i}{x_i}$:

$$P(x_1, \dots, x_k) = A(N,m) \cdot \prod_{i=1}^{k} \binom{N_i}{x_i}$$

Заміняємо `A` оберненою:

$$P(x_1, \dots, x_k) = \frac{1}{\binom{N}{m}} \cdot \prod_{i=1}^{k} \binom{N_i}{x_i}$$

Переносимо множник у знаменник дробу:

$$P(x_1, \dots, x_k) = \frac{\prod_{i=1}^{k} \binom{N_i}{x_i}}{\binom{N}{m}}$$

Всі біноміальні коефіцієнти ми обчислюємо через добуток:

$$C(n,r) = \binom{n}{r} = \prod_{t=1}^{\min(r,n-r)} \frac{n - \min(r, n-r) + t}{t}$$

Остаточна формула розподілу для $x_1, \dots, x_k$:

Модель дозволяє обчислити ймовірність будь-якої конкретної конфігурації.

$$P(x_1, \dots, x_k) = \frac{\prod_{i=1}^{k} C(N_i, x_i)}{C(N, m)}$$

2.9.1 Таблиця функцій алгоритму моделі

Мат модель	Опис	Програмна реалізація
$r' = min(r, n-r)$	Повертає r' який використовується у функції $C(n, r)$	`mirror_index`
$C(n, r) = \prod_{t=1}^{r'} \frac{n-r'+t}{t}$	Функція обчислює комбінацію через добуток, використовує `mirror_index`, для скорочення кроків.	`combination_by_product`
$P(x_1, \dots, x_k) = \frac{\prod_{i=1}^{k} C(N_i, x_i)}{C(N, m)}$	Загальна модель розподілу де $\sum_{i=1}^{k} N_i = N$ та $\sum_{i=1}^{k} x_i = m$ для однієї конкретної конфігурації $x_1, \dots, x_k$	`probability`

- `Probability`

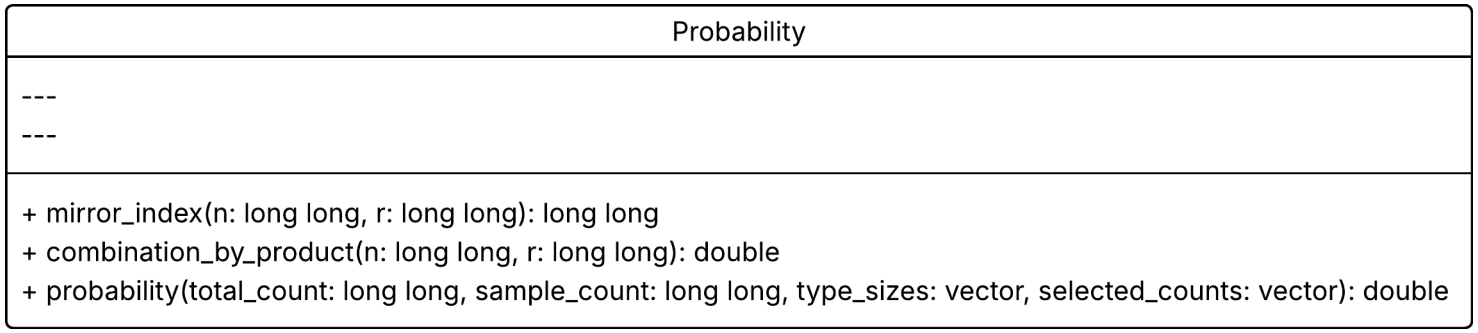

Рис. 2 Клас Probability

- `Probability::mirror_index`

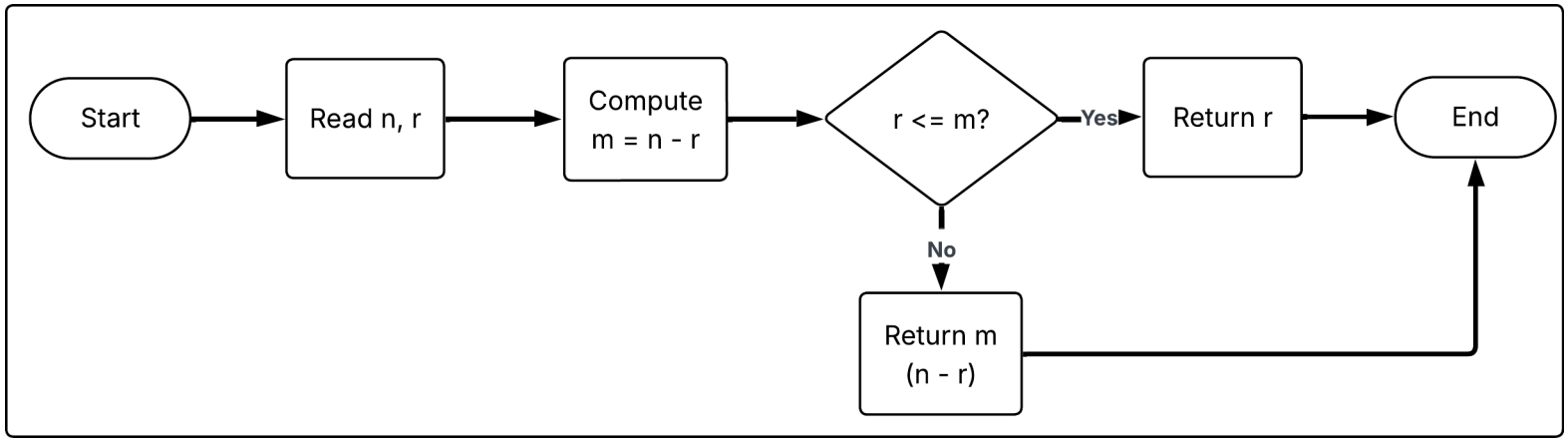

Рис. 3 Блок схема обчислення дзеркального індексу

- `Probability::combination_by_product`

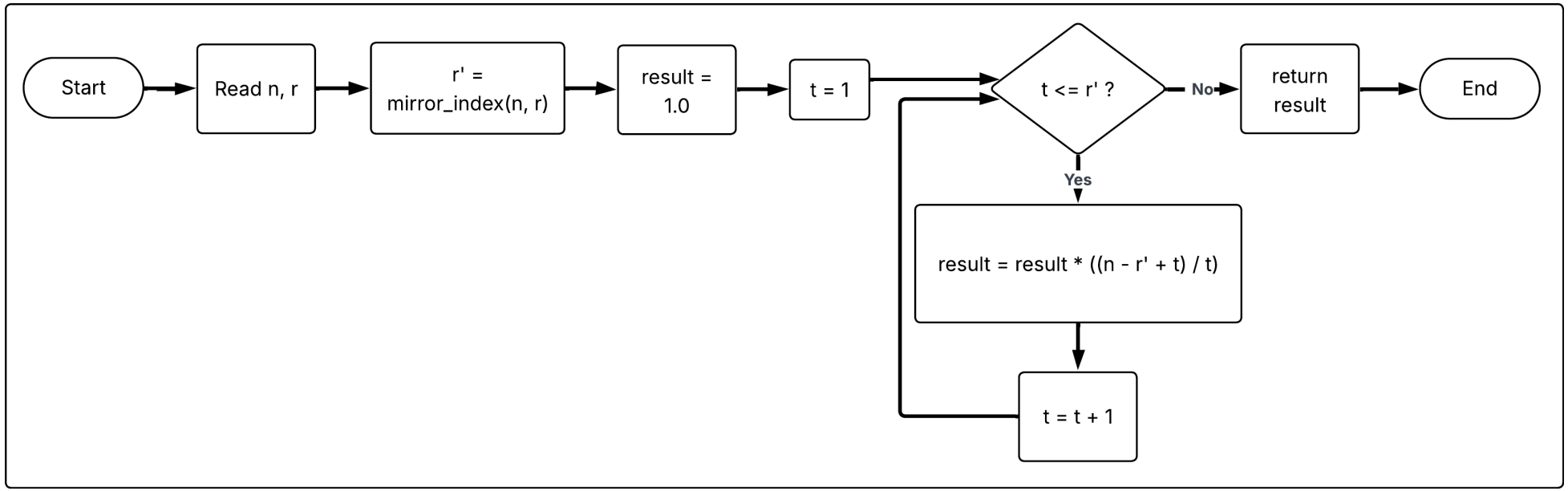

Рис. 4 Блок схема ітеративного обчислення біноміального коефіцієнта

- `Probability::probability`

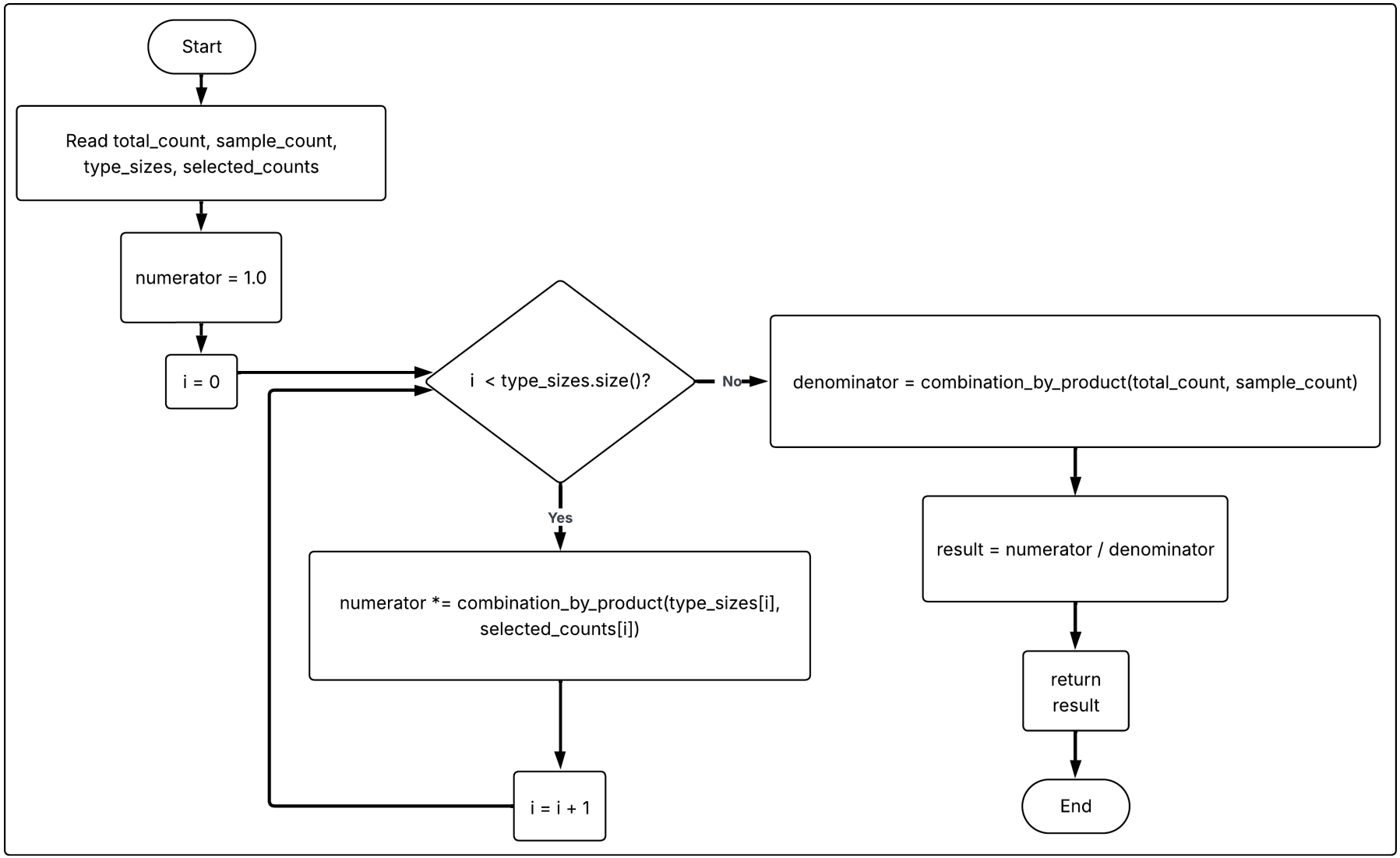

Рис. 5 Блок схема обчислення ймовірності за гіпергеометричним розподілом

Таблиця перевірок в `Validator`

№	Перевірка	Пояснення	Функція в коді
1	$N \geq 0$	Загальна кількість об'єктів не може бути від'ємною.	`validate_total_coun`
2	$k \geq 0$	Кількість типів об'єктів не може бути від'ємною.	`validate_type_count`
3	$N_i \geq 0,\ i = 1, \dots, k$	Кількість об'єктів кожного типу не може бути від'ємною.	`validate_type_sizes_non_negative`
4	$\sum_{i=1}^{k} N_i = N$	Сума кількостей об'єктів усіх типів повинна дорівнювати загальній кількості.	`validate_type_sizes_sum`
5	$0 \leq m \leq N$	Кількість вибраних об'єктів повинна бути в межах від нуля до загальної кількості.	`validate_sample_count`
6	$x_i \geq 0,\ i = 1, \dots, k$	Кількість вибраних об'єктів кожного типу не може бути від'ємною.	`validate_selected_counts_non_negative`
7	$x_i \leq N_i,\ i = 1, \dots, k$	Неможливо вибрати більше об'єктів певного типу, ніж існує у сукупності.	`validate_selected_not_greater_than_type_sizes`
8	$\sum_{i=1}^{k} x_i = m$	Сума вибраних об'єктів усіх типів повинна дорівнювати загальній кількості вибірки.	`validate_selected_counts_sum`
9	`\\|type_sizes\\|` = k	Кількість елементів у контейнері `type_sizes` повинна дорівнювати кількості типів `type_count`	`validate_type_sizes_count`

10	`|selected_counts|` = k	Кількість елементів у контейнері selected_counts також повинна дорівнювати `type_count`	`validate_selected_counts_count`

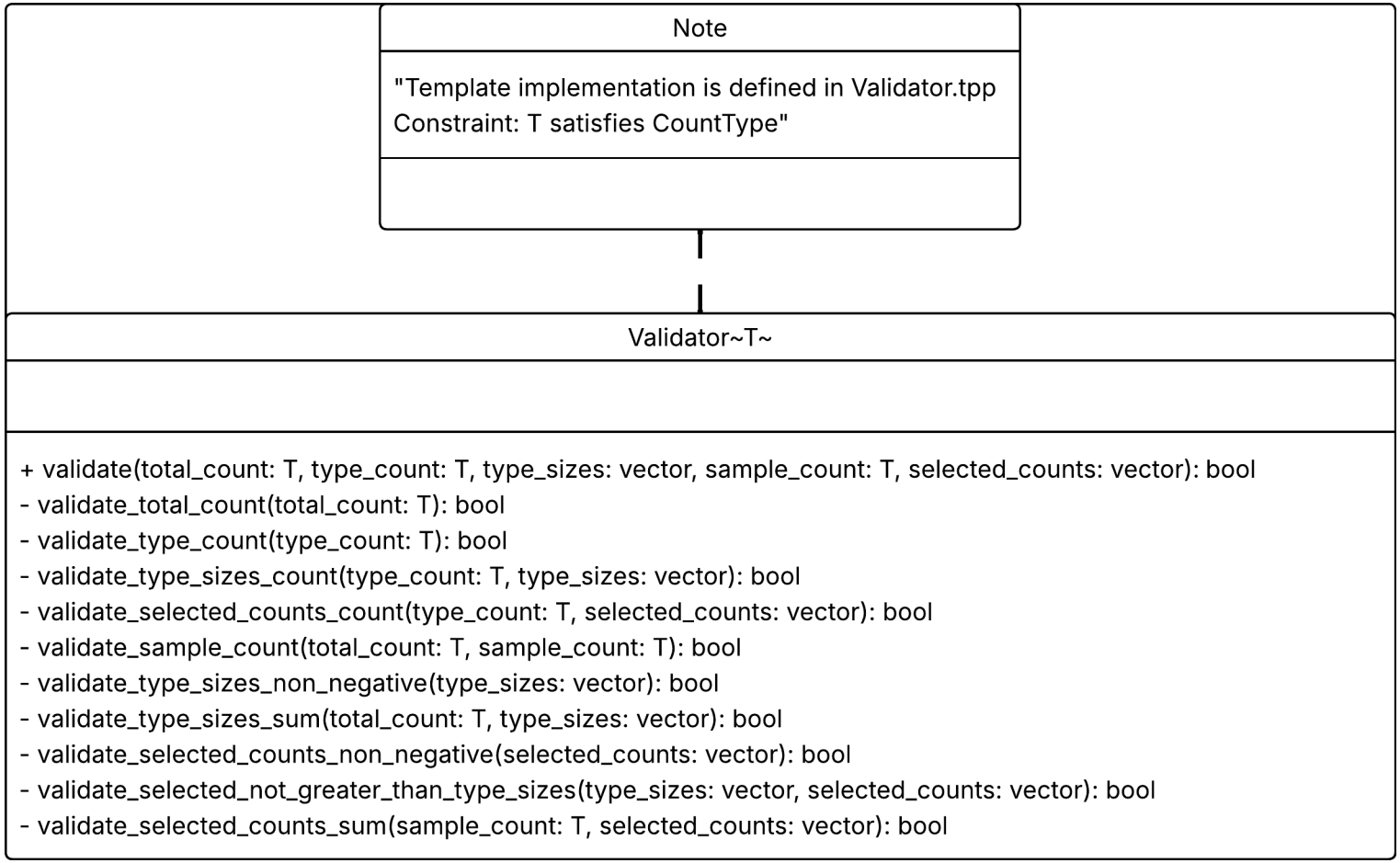

Рис. 6 Клас Validator

2.10 Обчислення ймовірності події.

Ймовірність події задачі з телевізорами `(39)` визначається як сума ймовірностей усіх конфігурацій, для яких кількість об'єктів першого типу у вибірці задовольняє умову $x_1 \geq 2$.

Маємо:

$$N_1 = 5, N_2 = 7, m = 4$$

та подію $x_1 \geq 2$

Тепер нам потрібно знайти всі допустимі конфігурації (x_1, x_2), для яких:

$$x_1 + x_2 = 4,\ 0 \leq x_1 \leq 5,\ 0 \leq x_2 \leq 7,\ x_1 \geq 2$$

Тобто ми повинні застосувати нашу модель для $(2, 2), (3, 1), (4, 0)$.

2.10.1 Допоміжні функції

Функція яка генерує конфігурації, і подія поверх неї "хоча б".

Функція configurations$(N_1, \dots, N_k, m, j, h)$ повертає всі вектори $(x_1, \dots, x_k)$ за умов:

$$\sum_{i=1}^{k} x_i = m$$

$$0 \leq x_i \leq N_i$$

$$x_j \geq h$$

- `Configurations::configurations`

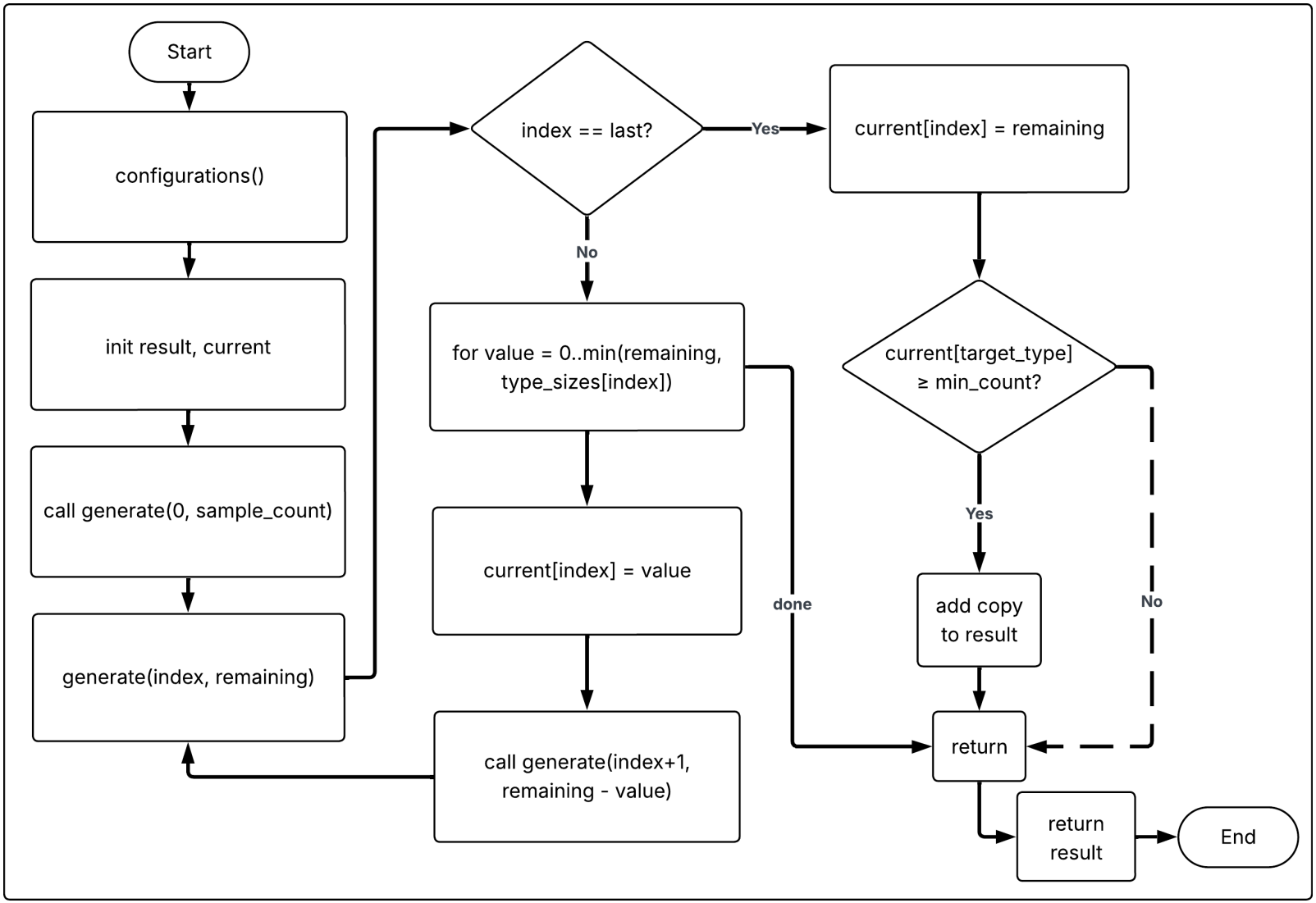

Рис. 7 Блок схема генерації конфігурацій вибірки за обмеженнями

І функція що обчислює суму всіх конфігурацій:

$$\text{sumprobability}(N_1, \dots, N_k, m, j, h) = \sum_{(x_1,\dots,x_k)\in\text{configurations}} P(x_1, \dots, x_k)$$

- `sum_probability`

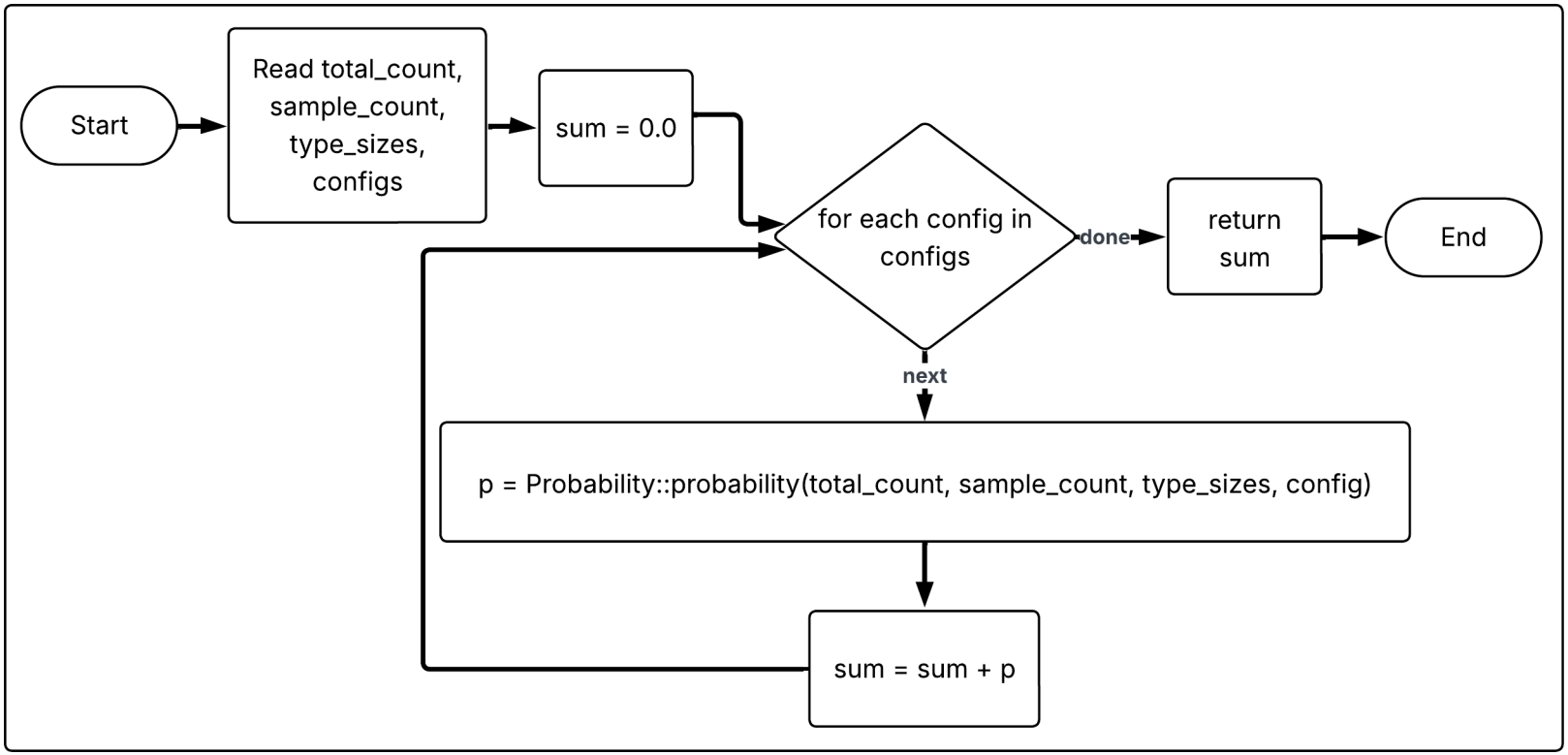

Рис. 8 Блок схема обчислення сумарної ймовірності за всіма конфігураціями

3 Тема 4. Bernoulli

3.1 Мета

- Навчитися застосовувати формулу Бернуллі для розв'язання практичних задач.

3.2 Теорія

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Формула Бернуллі або біноміальний розподіл описує схему, коли незалежне випробування повторюється n разів з однаковою ймовірністю успіху p. Коефіцієнт $\binom{n}{k}$ показує кількість різних способів вибрати k успіхів серед n випробувань. Множник p^k дає ймовірність того, що саме ці k випробувань будуть успішними, а множник $(1-p)^{n-k}$ дає ймовірність того, що решта $n-k$ випробувань будуть не успішними. Добуток цих множників описує одну конкретну конфігурацію успіхів і невдач, а множник $\binom{n}{k}$ враховує всі можливі перестановки таких конфігурацій.

Використовується коли кількість випробувань відносно невелика або коли потрібне точне значення ймовірності. Підходить для дискретних подій типу успіх або невдача. Класичні приклади це підкидання монети, дефектні вироби в партії або правильні відповіді у тесті.

3.3 Задача

Комутатор обслуговує 100 абонентів, ймовірність того, що на впродовж хвилини абонент зателефонує на комутатор дорівнює 0,01. Знайти ймовірність того, що впродовж хвилини:

- а) зателефонує три абоненти;
- б) зателефонує менше трьох абонентів;
- в) зателефонує більше трьох абонентів;
- г) зателефонує хоча б один абонент.

Підставимо:

$$P(X=k)=\binom{n}{k}p^k(1-p)^{n-k}$$

$n=100$	кількість абонентів
$p=0.01$	ймовірність що один абонент зателефонує за хвилину
k	кількість абонентів, які зателефонують за хвилину
C_{100}^k	кількість способів вибрати k абонентів із 100, які зроблять дзвінок
p^k	ймовірність того, що саме ці k абонентів зателефонують
$(1-p)^{100-k}$	ймовірність того, що решта абонентів не зателефонують
$P(X=k)$	ймовірність того, що рівно k абонентів зателефонують за хвилину

$$P(X=k)=C_{100}^k(0.01)^k(0.99)^{100-k}$$

Тоді:

а) зателефонує три абоненти:

$$P(X=3)=C_{100}^3(0.01)^3(0.99)^{100-3}$$

б) зателефонує менше трьох абонентів:

$$P(X<3)=P(X=0)+P(X=1)+P(X=2)$$
$$P(X<3)=C_{100}^0(0.01)^0(0.99)^{100}+C_{100}^1(0.01)^1(0.99)^{99}+C_{100}^2(0.01)^2(0.99)^{98}$$

в) зателефонує більше трьох абонентів:

$$P(X > 3) = 1 - (P(X = 0) + P(X = 1) + P(X = 2) + P(X = 3))$$
$$P(X > 3) = 1 - (C_{100}^{0}(0.01)^{0}(0.99)^{100} + C_{100}^{1}(0.01)^{1}(0.99)^{99} + C_{100}^{2}(0.01)^{2}(0.99)^{98} + C_{100}^{3}(0.01)^{3}(0.99)^{97})$$

г) зателефонує хоча б один абонент

$$P(X \geq 1) = 1 - C_{100}^{0}(0.01)^{0}(0.99)^{100}$$

3.4 Функція вибору стратегії обчислення

Функція приймає n, k, p, і тип запиту $mode$ - це символьний код режиму:

```
double binomial_probability(long long n, long long k, double p,
    char mode   // '=', '<', '>', 'a'
)
```

$mode$	логіка	функція
`=`	рахується один вираз $P(X = k)$	`probability_equal(k, single)`
`<`	сумуються $P(X = 0) + \cdots + P(X = k - 1)$	`probability_less(k, single)`
`>`	використовується доповнення $1 - \sum_{i=0}^{k} P(X = i)$	`probability_greater(k, single)`
`a`	використовується $1 - P(X = 0)$	`probability_at_least_one(single)`

Де `single` це функція що обчислює один член біноміальної формули ймовірності:

`combination_by_product` це ефективне обчислення комбінації, реалізація якого в роботі 2026-03-08-probability-combination

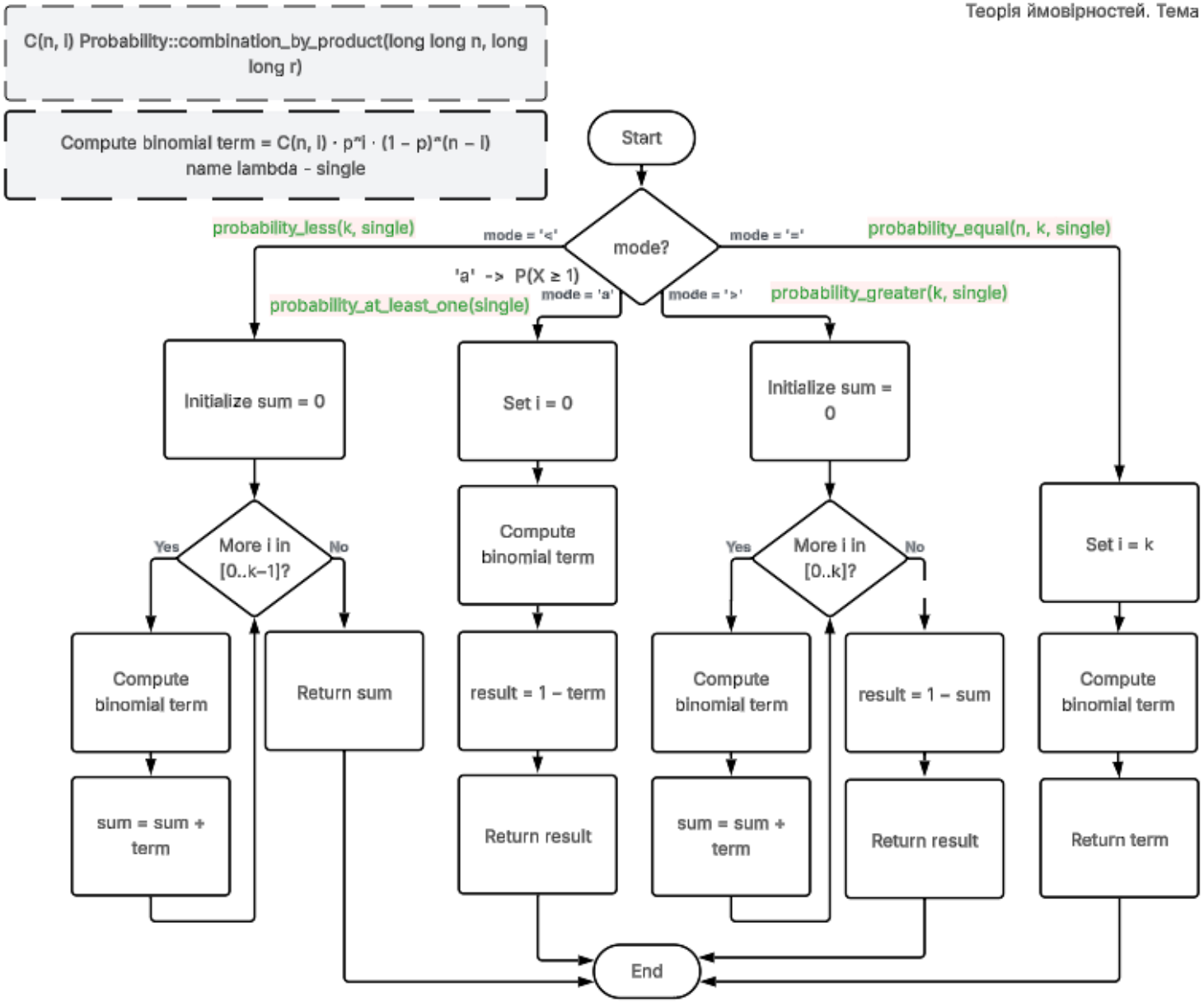

Рис. 9 Блок схема обчислення ймовірностей біноміального розподілу для різних режимів

4 Практичне заняття №5. ЗМІ. Тема 4. Laplace, Poisson

4.1 Мета

- Навчитися застосовувати формулу Пуассона, Лапласа для розв'язання практичних задач. Зробити порівняння Бернуллі, Пуассон, Лаплас.
- Приклад реалізації Бернуллі був виконаний окремо

4.2 Теорія

4.3 Локальна теорема Лапласа

$$P(X = k) \approx \frac{1}{\sqrt{npq}}\varphi(x)$$

де:

$$x = \frac{k - np}{\sqrt{npq}}, \qquad q = 1 - p, \qquad \varphi(x) = \frac{1}{\sqrt{2\pi}}e^{-x/2}$$

якщо підставити:

$$P(X = k) \approx \frac{1}{\sqrt{npq}} \cdot \frac{1}{\sqrt{2\pi}}e\left(-\frac{(k - np)^2}{2npq}\right), \qquad q = 1 - p$$

Локальна теорема Лапласа є наближенням біноміального розподілу нормальним розподілом. Вираз np є математичним сподіванням кількості успіхів, а npq є дисперсією. Значення $x = \frac{k-np}{\sqrt{npq}}$ нормалізує відхилення фактичного значення k від середнього. Функція $\varphi(x)$ є щільністю стандартного нормального розподілу і дає значення ймовірності після масштабування множником $\frac{1}{\sqrt{npq}}$.

Використовується коли n велике і p не дуже мале. При великому n біноміальний розподіл можна наближено замінити нормальним. Метод дозволяє швидко оцінювати ймовірності без обчислення великих комбінацій.

4.4 Інтегральна теорема Лапласа

$$P(k_1 \leq X \leq k_2) \approx \Phi(x_2) - \Phi(x_1)$$

де:

$$x_1 = \frac{k_1 - np}{\sqrt{npq}}, \qquad x_2 = \frac{k_2 - np}{\sqrt{npq}}, \qquad q = 1 - p$$

$$\Phi(x) = \int_0^x \frac{1}{\sqrt{2pi}} e^{-t^2/2} dt$$

Значення np є математичним сподіванням біноміального розподілу і показує середню кількість успіхів. Величина $\sqrt{npq}$ є стандартним відхиленням і визначає масштаб розсіювання значень. Межі k_1 k_2 переводяться у нормовані координати x_1 і x_2, які показують відхилення від середнього у одиницях стандартного відхилення. Функція $\Phi(x)$ є інтегральною функцією стандартного нормального розподілу і дає накопичену ймовірність від нуля до точки x. Різниця $\Phi(x_2) - \Phi(x_1)$ дає ймовірність того, що нормальна випадкова величина лежить у відповідному інтервалі.

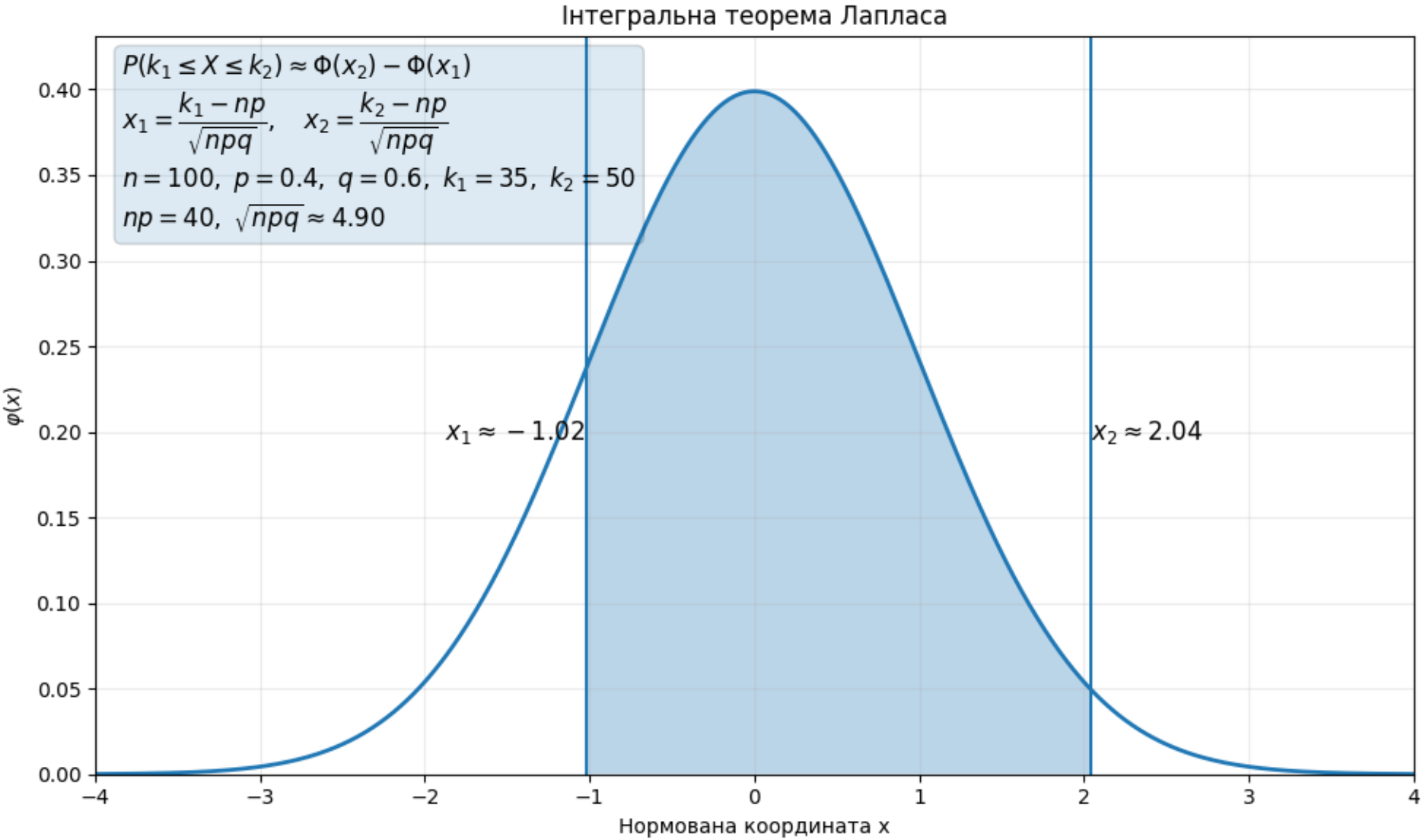

Рис. 10 Пояснювальний графік для інтегральної теореми Лапласа.

На графіку показано стандартну нормальну криву $\varphi(x)$, межі x_1 і x_2, а також заштриховану область, яка відповідає ймовірності

$$P(k_1 \leq X \leq k_2) \approx \Phi(x_2) - \Phi(x_1)$$

У прикладі взято $n = 100, \quad p = 0.4, \quad q = 0.6, \quad k_1 = 35, \quad k2 = 50$

тому: $np = 40, \qquad \sqrt{npq} = 4.9$

$$x_1 = \frac{k_1 - np}{\sqrt{npq}} = \frac{35 - 40}{4.9} \approx -1.02$$

$$x_2 = \frac{k_2 - np}{\sqrt{npq}} = \frac{50 - 40}{4.9} \approx 2.04$$

Сам інтеграл показує зміст під кривою, нормуємо межі інтервалу і отримуємо x_1 та x_2, у таблиці знаходимо $\Phi(x_1)$ і $\Phi(x_2)$, віднімаємо їх.

Інтегральна теорема Лапласа використовується для наближеного обчислення ймовірності того, що біноміальна випадкова величина потрапляє у певний інтервал значень. На відміну від локальної теореми, тут розглядається не одна точка $X = k$, а діапазон значень. У задачах статистики це використовується для оцінки інтервалів і відхилень від середнього.

4.5 Пуасона

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad \lambda = np$$

Розподіл Пуассона є граничним випадком біноміального розподілу, коли кількість випробувань дуже велика, а ймовірність успіху дуже мала. Параметр λ означає середню кількість появ події у всій серії випробувань. Множник

λ^k враховує кількість появ події, $k!$ компенсує кількість можливих перестановок подій, а коефіцієнт $e^{-\lambda}$ робить суму всіх значень розподілу рівною 1.

Використовується для рідкісних подій. Типові приклади це кількість відмов обладнання, кількість дзвінків у кол центр за короткий час або кількість дефектів на великій площі матеріалу.

4.6 Загальна порівняльна характеристика

Характеристика	Бернуллі	Локальна теорема Лапласа	Інтегральна теорема Лапласа	Пуассона
Тип моделі	Точний біноміальний розподіл	Нормальне наближення	Нормальне наближення	Граничне наближення
Тип змінної	Дискретна	Неперервна модель	Неперервна модель	Дискретна
Що обчислює	Ймовірність $X = k$	Ймовірність $X = k$	Ймовірність інтервалу $k_1 \leq X \leq k_2$	Ймовірність $X = k$
Умови застосування	n будь яке	n велике, p не дуже мале	n велике, p не дуже мале	n велике, p мале
Основний параметр	n, p	np, npq	np, npq	$\lambda = np$
Тип задач	Точні ймовірності	Наближення для одного значення	Наближення для інтервалу	Рідкісні події

4.7 Завдання 47

Застосовуючи

- а) формулу Бернуллі;

- б) локальну теорему Лапласа
- в) формулу Пуассона

знайти ймовірність того, що серед 200 осіб виявиться четверо шульгів, якщо у середньому шульги становлять 1%.

4.7.1 Таблична функція Лапласа

Функція для обчислення значень, які беруться з таблиці Лапласа:

$$\Phi(x) = \int_0^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt$$

яка виражається через функцію помилки так:

$$\Phi(x) = \frac{1}{2}\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)$$

```
double laplace_phi(double x)
{ return 0.5 * std::erf(x / std::sqrt(2.0)); }
```

4.7.2 Щільність нормального розподілу

Функція використовується в локальній теоремі Лапласа

$$\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x/2}$$

```
double laplace_local_phi(const double x)
{ return (1.0 / std::sqrt(2.0 * M_PI)) * std::exp(-x * x / 2.0); }
```

4.7.3 UML diagrams: Bernoulli, Laplace, Poisson

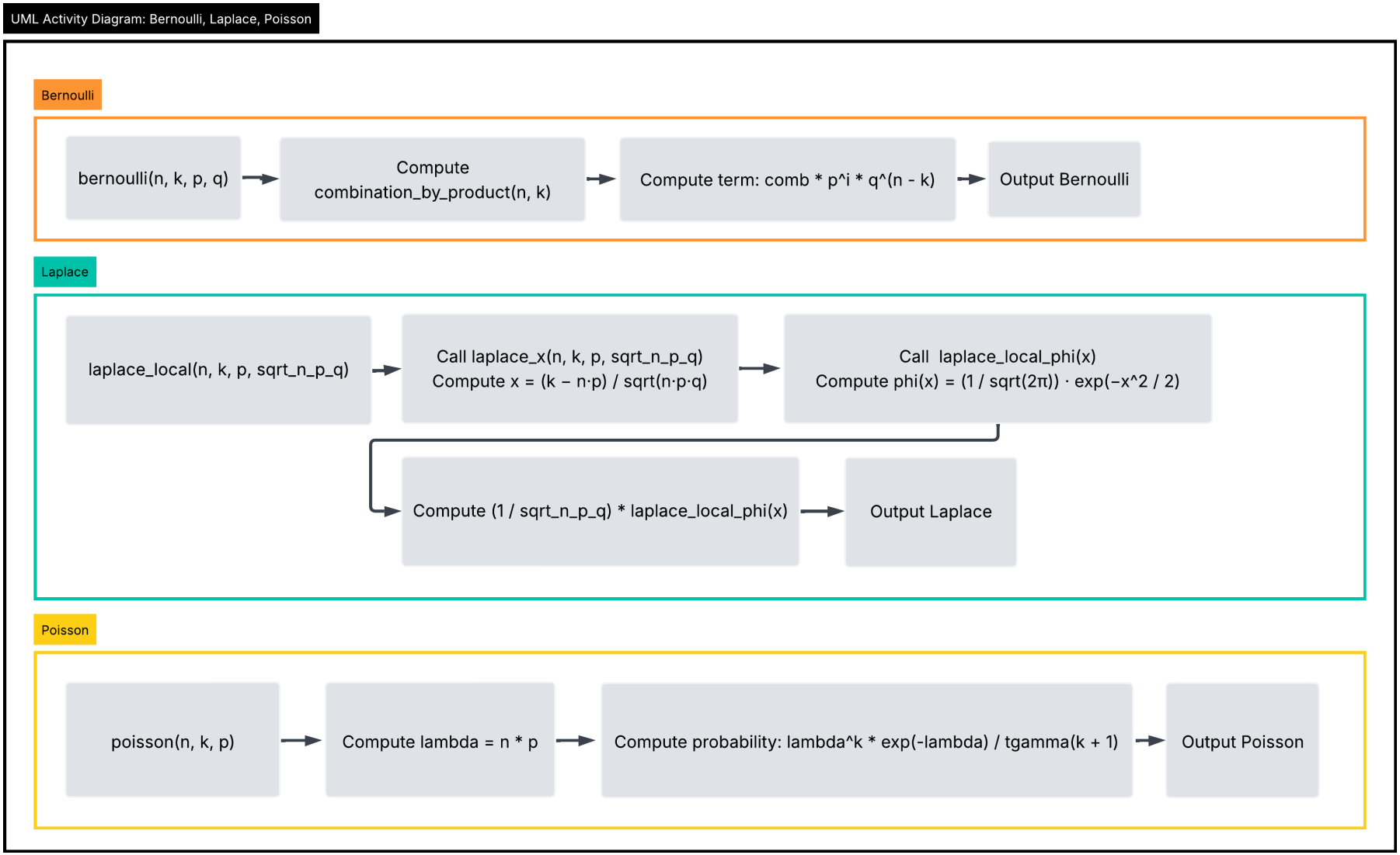

Рис. 11 Blank diagram - UML Activity Diagrams Bernoulli, Laplace, Poisson

Приклад обчислень для завдання 47:

```
/home/tysser/Scripts/jetbrains/cpp_course/cmake-build-debug/math/III_course/2026
Введіть загальну кількість випробувань n = 200
Введіть ймовірність p = 0.01
Введіть очікувану кількість успіхів k = 4
==============================================================================
P(X = 4) для 200 випробувань при ймовірності успіху p = 0.010000
------------------------------------------------------------------------------
Bernoulli: 0.090220
Laplace:   0.103251
Poisson:   0.090224
==============================================================================
```

Рис. 12 Результат обчислення функцій

4.8 Висновок

Формула Бернуллі дає точне значення ймовірності $P(X = k)$ і застосовується безпосередньо для обчислення біноміальної ймовірності. Локальна теорема Муавра Лапласа використовується для наближеного обчислення $P(X = k)$ при великому числі випробувань n і достатньо великому значенні дисперсії біноміального розподілу npq. Якщо ймовірність успіху p дуже мала або дисперсія біноміального розподілу є малою, точність нормального наближення погіршується. Інтегральна теорема Муавра Лапласа застосовується для знаходження ймовірності того, що число успіхів лежить у певному інтервалі. Вона також використовується за умови великої кількості випробувань і достатньо великої дисперсії біноміального розподілу. Формула Пуассона використовується для рідкісних подій, коли ймовірність успіху p в одному випробуванні мала, кількість випробувань велика, а

математичне сподівання числа успіхів np має скінченне значення. У розглянутій задачі саме наближення Пуассона дає результат, найближчий до точного значення, отриманого за формулою Бернуллі.

---

5 Практичне заняття №7. ЗМІ. Тема 4

5.1 Числові характеристики дискретної випадкової величини

Відомий закон розподілу випадкової величини X. Знайти її математичне сподівання, дисперсію та середнє квадратичне відхилення.

```
std::vector<distribution> tasks = {
    {"7.1",  {-2, 1, 3, 5},  {0.2, 0.28, 0.42, 0.1}},
    {"7.2",  {-5, 2, 3, 4},  {0.4, 0.29, 0.11, 0.2}},
    {"7.3",  {-3, -2, 1, 3}, {0.35, 0.15, 0.2, 0.3}},
    {"7.4",  {-4, 0, 2, 5},  {0.1, 0.2, 0.25, 0.45}},
    {"7.5",  {-2, 2, 5, 8},  {0.27, 0.21, 0.22, 0.3}},
    {"7.6",  {-1, 1, 5, 7},  {0.2, 0.33, 0.24, 0.23}},
    {"7.7",  {-2, -1, 3, 5}, {0.22, 0.3, 0.21, 0.27}},
    {"7.8",  {-5, 2, 3, 4},  {0.2, 0.28, 0.42, 0.1}},
    {"7.9",  {-3, -2, 1, 3}, {0.4, 0.29, 0.1, 0.21}},
    {"7.10", {-4, 0, 2, 5},  {0.35, 0.15, 0.2, 0.3}},
    {"7.11", {-2, 2, 5, 8},  {0.1, 0.2, 0.25, 0.45}},
    {"7.12", {-1, 1, 5, 7},  {0.27, 0.21, 0.22, 0.3}},
    {"7.13", {-2, 3, 4, 5},  {0.15, 0.25, 0.25, 0.35}},
    {"7.14", {-5, -2, 3, 4}, {0.1, 0.23, 0.22, 0.45}},
    {"7.15", {-4, 1, 2, 5},  {0.13, 0.21, 0.22, 0.44}},
    {"7.16", {-2, 2, 4, 6},  {0.14, 0.26, 0.25, 0.35}},
    {"7.17", {-3, -2, 2, 6}, {0.21, 0.22, 0.27, 0.3}},
```

```
    {"7.18", {-1, 3, 5, 6},  {0.18, 0.21, 0.23, 0.38}},
    {"7.19", {-5, -1, 3, 5}, {0.21, 0.22, 0.25, 0.32}},
    {"7.20", {-2, -1, 4, 9}, {0.2, 0.28, 0.42, 0.1}},
    {"7.21", {-5, -2, -1, 4},{0.4, 0.29, 0.1, 0.21}},
    {"7.22", {-4, -1, 0, 5}, {0.35, 0.15, 0.2, 0.3}},
    {"7.23", {-1, 2, 4, 7},  {0.1, 0.2, 0.25, 0.45}},
    {"7.24", {-5, -3, -2, 1},{0.2, 0.31, 0.22, 0.27}},
    {"7.25", {-2, 2, 5, 6},  {0.25, 0.22, 0.23, 0.3}},
    {"7.26", {-4, 2, 4, 6},  {0.22, 0.2, 0.23, 0.35}},
    {"7.27", {-3, -2, 1, 6}, {0.2, 0.23, 0.32, 0.25}},
    {"7.28", {-4, 0, 2, 5},  {0.3, 0.32, 0.22, 0.16}},
    {"7.29", {-1, 3, 5, 8},  {0.25, 0.4, 0.2, 0.15}},
    {"7.30", {-3, 1, 3, 5},  {0.05, 0.25, 0.52, 0.18}}
};
```

5.2 Математичне сподівання

$$M(X) = \sum x_i p_i$$

```
double expectation(const std::vector<int>& x,
                   const std::vector<double>& p)
{
    double sum = 0.0;

    for(size_t i = 0; i < x.size(); ++i)
        sum += x[i] * p[i];

    return sum;
}
```

5.3 Квадрат очікування

$$\sum x_i^2 p_i$$

```
double expectation_square(const std::vector<int>& x,
                          const std::vector<double>& p)
{
    double sum = 0.0;

    for(size_t i = 0; i < x.size(); ++i)
        sum += x[i] * x[i] * p[i];

    return sum;
}
```

5.4 Дисперсія

$$D(X) = \sum x_i^2 p_i - (M(X))^2$$

```
double variance(const std::vector<int>& x,
                const std::vector<double>& p)
{
    double m = expectation(x, p);
    double m2 = expectation_square(x, p);

    return m2 - m * m;
}
```

5.5 Середнє квадратичне відхилення

$$\sigma = \sqrt{D(X)}$$

```
double standard_deviation(const std::vector<int>& x,
             const std::vector<double>& p)
{
   return std::sqrt(variance(x, p));
}
```

5.6 Результат виконання

Номер	M(X)	D(X)	Sigma
7.1	1.6400	4.6704	2.1611
7.2	-0.2900	15.2659	3.9072
7.3	-0.2500	6.5875	2.5666
7.4	2.3500	8.3275	2.8857
7.5	3.3800	15.1956	3.8982
7.6	2.9400	9.1564	3.0260
7.7	1.2400	8.2824	2.8779
7.8	1.2200	10.0116	3.1641
7.9	-1.0500	5.6475	2.3764
7.10	0.5000	13.6500	3.6946
7.11	5.0500	10.7475	3.2783
7.12	3.1400	10.8204	3.2894
7.13	3.2000	5.3600	2.3152
7.14	1.5000	10.3500	3.2171
7.15	2.3300	8.7411	2.9565
7.16	3.3400	7.0444	2.6541
7.17	1.2700	13.0371	3.6107

7.18	3.8800	6.4456	2.5388
7.19	1.0800	14.5536	3.8149
7.20	1.9000	12.2900	3.5057
7.21	-1.8400	11.2344	3.3518
7.22	-0.0500	13.2475	3.6397
7.23	4.4500	7.1475	2.6735
7.24	-2.1000	4.5300	2.1284
7.25	2.8900	10.0779	3.1746
7.26	2.5400	14.1484	3.7614
7.27	0.7600	11.4624	3.3856
7.28	0.0400	9.6784	3.1110
7.29	3.1500	8.5275	2.9202
7.30	2.5600	3.3264	1.8238

6 Випадкові величини і процеси. Дискретні випадкові величини

Закон розподілу дискретної випадкової величини

ТІМС-ЛПР-06+

6.1 Завдання 6.1

6.1–6.6 Маємо n ламп, кожна з яких з ймовірністю p є якісною. Лампа вгвинчується в прилад і вмикається струм. При вмиканні струму дефектна лампа одразу виходить з ладу, після чого замінюється іншою. Розглядається випадкова величина X - число випробуваних ламп. Знайти її закон розподілу.

Варіант	6.1	6.2	6.3	6.4	6.5	6.6
n	5	4	5	4	5	4

Варіант	6.1	6.2	6.3	6.4	6.5	6.6
p	0.9	0.7	0.8	0.8	0.7	0.9

Розв'язання варіант 6.1:

$n = 5, p = 0.9, q = 1 - p = 0.1$

q - ймовірність того, що лампа дефектна.

Випадкова величина X - кількість перевірених ламп до появи першої якісної або до вичерпання всіх ламп. Закон розподілу має геометричний характер і задається формулою:

$$P(X = k) = \begin{cases} q^{k-1}p, & k = 1, 2, \dots, n-1 \\ q^{n-1}, & k = n \end{cases}$$

Ця формула означає, що для значення $X = k$ перші $k - 1$ ламп є дефектними, а k-та лампа якісна.

Обчислимо ймовірності для всіх можливих значень випадкової величини X:

$P(X = 1) = 0.9$

$P(X = 2) = 0.1 \cdot 0.9 = 0.09$

$P(X = 3) = 0.1^2 \cdot 0.9 = 0.01 \cdot 0.9 = 0.009$

$P(X = 4) = 0.1^3 \cdot 0.9 = 0.001 \cdot 0.9 = 0.0009$

$P(X = 5) = 0.1^4 = 0.0001$

$0.9 + 0.09 + 0.009 + 0.0009 + 0.0001 = 1$

Розподіл побудовано правильно.

Для автоматизації обчислень реалізуємо програмну функцію, яка формує закон розподілу для заданих n та p

```
std::vector<double> geometric_distribution(int n, double p)
```

Для кожного можливого значення випадкової величини X обчислено відповідну ймовірність $P(X = k)$. Праворуч наведено візуальне подання процесу перевірки ламп: 0 означає дефектну лампу, 1 означає першу якісну лампу, символ x позначає лампи, які вже не перевірялися.

```
/home/tysser/Scripts/jetbrains/cpp_course/cmake-
Variant 6.1
    P(X=1) =   0.900000    1 x x x x
    P(X=2) =   0.090000    0 1 x x x
    P(X=3) =   0.009000    0 0 1 x x
    P(X=4) =   0.000900    0 0 0 1 x
    P(X=5) =   0.000100    0 0 0 0 1
```

Рис. 13 img.png

6.2 Завдання 6.7

6.7-6.12 Випробовується пристрій, який складається з трьох незалежно працюючих приладів. Ймовірності відмови приладів p_1, p_2, p_3. Розглядається випадкова величина X - число приладів, які вийшли з ладу. Знайти її закон розподілу.

Варіант	6.7	6.8	6.9	6.10	6.11	6.12
p_1	0.5	0.6	0.7	0.3	0.4	0.3
p_2	0.6	0.7	0.8	0.8	0.7	0.6

Варіант	6.7	6.8	6.9	6.10	6.11	6.12
p_3	0.7	0.8	0.5	0.6	0.5	0.4

Розв'язання варіант 6.7:

Ймовірності відмови приладів: $p_1 = 0.5, p_2 = 0.6, p_3 = 0.7$

Ймовірності справної роботи: $1 - p_1 = 0.5, 1 - p_2 = 0.4, 1 - p_3 = 0.3$

Випадкова величина X це число приладів, які вийшли з ладу, тому $X \in \{0, 1, 2, 3\}$

$$P(X = k) = \sum_{\substack{A \subset \{1,2,3\} \\ |A| = k}} \left(\prod_{i \in A} p_i \right) \left(\prod_{j \notin A} (1 - p_j) \right)$$

Обчислюємо:

$P(X = 0) = (1 - p_1)(1 - p_2)(1 - p_3) = 0.5 \cdot 0.4 \cdot 0.3 = 0.06$

$P(X = 1) = p_1(1 - p_2)(1 - p_3) + (1 - p_1)p_2(1 - p_3) + (1 - p_1)(1 - p_2)p_3$

$P(X = 1) = 0.5 \cdot 0.4 \cdot 0.3 + 0.5 \cdot 0.6 \cdot 0.3 + 0.5 \cdot 0.4 \cdot 0.7$

$P(X = 1) = 0.06 + 0.09 + 0.14 = 0.29$

$P(X = 2) = p_1 p_2(1 - p_3) + p_1(1 - p_2)p_3 + (1 - p_1)p_2 p_3$

$P(X = 2) = 0.5 \cdot 0.6 \cdot 0.3 + 0.5 \cdot 0.4 \cdot 0.7 + 0.5 \cdot 0.6 \cdot 0.7$

$P(X = 2) = 0.09 + 0.14 + 0.21 = 0.44$

$P(X = 3) = p_1 p_2 p_3 = 0.5 \cdot 0.6 \cdot 0.7 = 0.21$

$0.06 + 0.29 + 0.44 + 0.21 = 1$

Розподіл побудовано правильно.

```
std::vector<double> failure_distribution(const std::vector<double>& p)
```

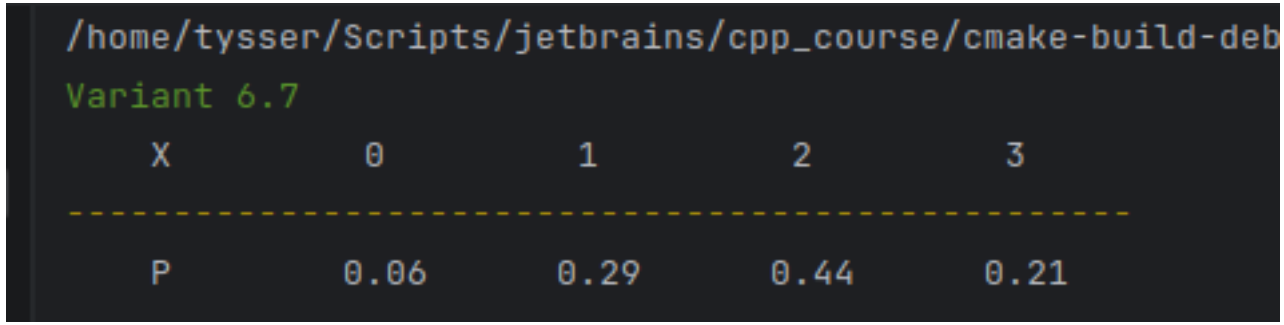

Рис. 14 img_2.png

6.3 Висновок

У роботі було знайдено закони розподілу дискретних випадкових величин для двох типів випадкових процесів. Для задачі з лампами отримано геометричний закон розподілу, а для задачі з трьома незалежними приладами закон розподілу побудовано шляхом перебору всіх можливих комбінацій відмов. В усіх випадках перевірено, що сума ймовірностей дорівнює одиниці, тому результати є коректними. Програмна реалізація підтвердила правильність теоретичних обчислень.

---

7 Випадкові величини і процеси. Неперервні випадкові величини

Закон розподілу неперервної випадкової величини

ТІМС-ЛПР-08+

7.1 Завдання 8

$8.1 - 8.30$ Випадкова величина X має функцію розподілу $F(x)$. Знайти:

а) ймовірність того, що X набуде значення з інтервалу $$;$$

б) щільність розподілу;

в) математичне сподівання, дисперсію та середнє квадратичне відхилення.

Побудувати графіки функції розподілу та щільності розподілу.

Варіант 8.2

$$F(x) = \begin{cases} 0, & x \leq 1 \\ \dfrac{x^2 - x}{2}, & 1 < x \leq 2 \\ 1, & x > 2 \end{cases}$$

$\alpha = -1; \beta = 1.5$

а) Ймовірність того, що випадкова величина набуде значення з інтервалу $[\alpha, \beta]$ обчислюється через функцію розподілу:

$P(\alpha \leq X \leq \beta) = F(\beta) - F(\alpha)$

Для цього використовується функція:

```
double probability(const distribution_case& c);
```

`distribution_case` тримає параметри конкретної задачі:

```
struct distribution_case {
    std::string label;
    double a;
```

```
    double b;
    double alpha;
    double beta;
    std::function<double(double)> distribution_function;
    std::function<double(double)> density_function;
};
```

`a, b` інтервал, де визначена випадкова величина

`alpha, beta` інтервал, для якого рахується ймовірність

`distribution_function` функція розподілу

`density_function` щільність розподілу, тобто похідна функції розподілу $f(x) = F'(x)$

Репозиторій `DistributionRepository` тримає всі завдання 8.1 - 8.2, і має два методи для відображення графіків

Для функції розподілу

```
void plot_distribution_function(std::size_t variant_number) const;
```

Для щільності розподілу

```
void plot_density_function(std::size_t variant_number) const;
```

Інші значення вираховуються методами класу `Statistics`

7.2 Клас Statistics

Формула чисельного інтегрування за методом Сімпсона.

```
double Statistics::integrate(Function f, double a, double b);
```

Інтервал $[a, b]$ розбивається на n рівних частин. На кожній парі відрізків функція апроксимується квадратичною параболою. Значення функції у вузлах беруться з різними вагами: коефіцієнт 4 для непарних точок, 2 для парних,

1 для крайніх. Має високу точність, тому що враховує кривизну функції а не площу прямокутників.

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{k=1\\ k\text{ odd}}}^{n-1} f(x_k) + 2\sum_{\substack{k=2\\ k\text{ even}}}^{n-2} f(x_k) + f(x_n)\right]$$

$$h = \frac{b-a}{n}, \quad x_k = a + kh$$

7.2.1 Математичне сподівання

```
double Statistics::expectation(Function f, double a, double b);
```

Середнє значення випадкової величини. Воно показує, навколо якого значення зосереджена випадкова величина при великій кількості спостережень.

$$\mathbb{E}[X] = \int_a^b x f(x)\,dx$$

7.2.2 Математичне сподівання квадрата

```
double Statistics::second_moment(Function f, double a, double b);
```

Допоміжна величина, яка характеризує розподіл значень з урахуванням їх квадрата. Вона використовується для обчислення дисперсії.

$$\mathbb{E}[X^2] = \int_a^b x^2 f(x)\,dx$$

7.2.3 Дисперсія

```
double Statistics::variance(Function f, double a, double b);
```

Міра розкиду значень випадкової величини відносно її середнього. Якщо дисперсія мала, значення зосереджені близько до математичного сподівання.

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

7.2.4 Середнє квадратичне відхилення

```
double Statistics::standard_deviation(Function f, double a, double b);
```

Це корінь із дисперсії. Показує типове відхилення значень від середнього.

$$\sigma = \sqrt{\mathrm{Var}(X)}$$

Результат виконання:

```
Task: 8.2
P(alpha <= X <= beta) = 0.375
M(X) = 1.58333
D(X) = 0.0763889
sigma(X) = 0.276385
```

Рис. 15 Завдання 8.2

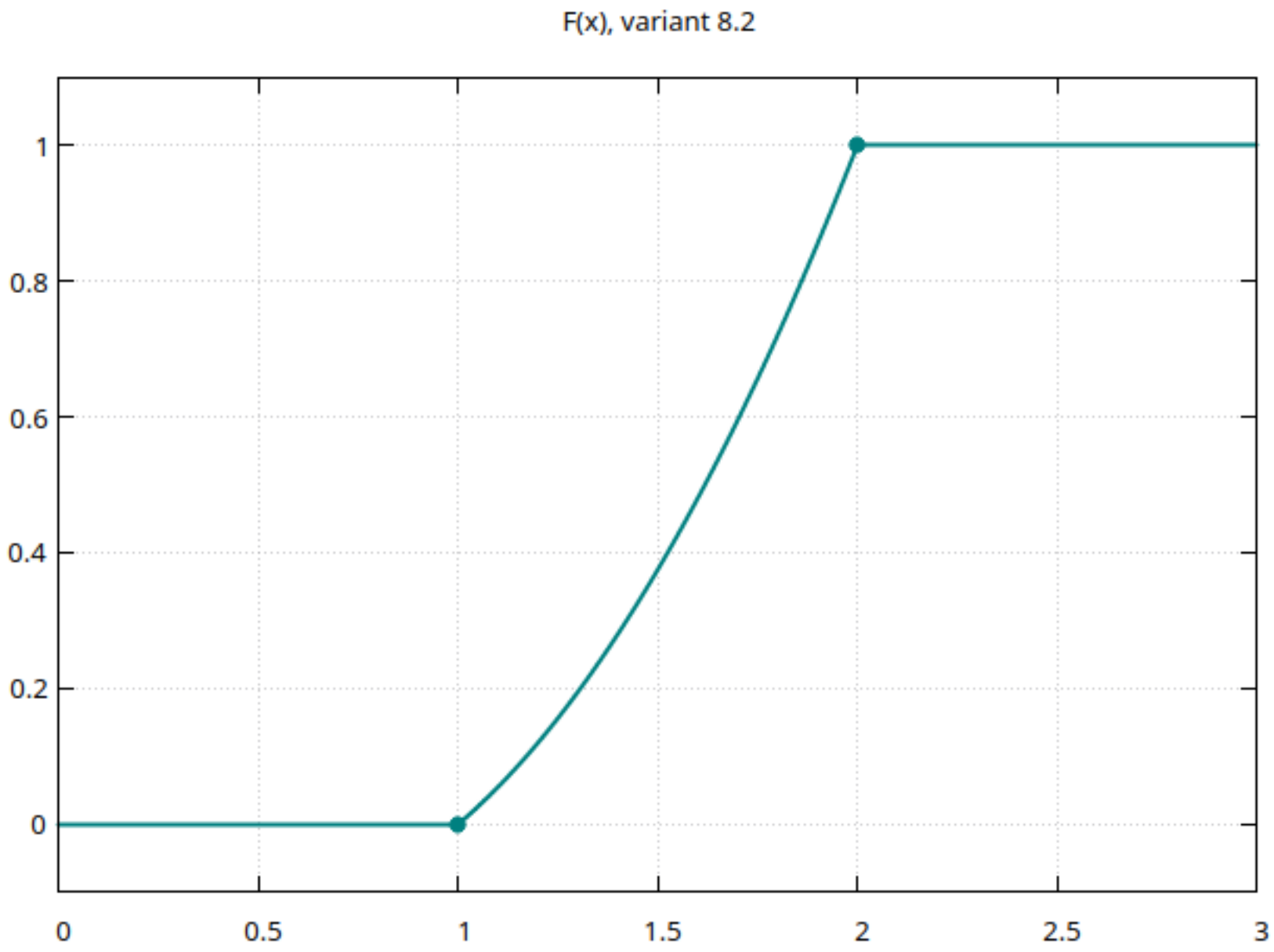

Рис. 16 Розподіл F(x)

```
/home/tysser/Scripts/jetbrains/cpp_course/cmake-build-deb
Variant 6.7
    X          0          1          2          3
-----------------------------------------------
    P         0.06       0.29       0.44       0.21
```

Рис. 17 Щільність розподілу f(x)

7.3 Додаток

Якщо є конкретна вибірка значень $x_1, ..., x_n$, формули описані нижче застосовуються напряму без інтегрування. Це класичний підхід математичної статистики. Інтегрування використовується тоді, коли є лише аналітична щільність розподілу $f(x)$ або теоретична модель випадкової величини. Тоді характеристики визначаються через інтеграли, наприклад $E[X] = \int xf(x)dx$. Якщо ж дані емпіричні, інтегрування є зайвим і навіть гіршим за точністю підходом, оскільки додає чисельну похибку.

7.3.1 Вибіркове середнє

Обчислює вибіркове середнє для емпіричної вибірки як оцінку математичного сподівання випадкової величини. Перевантажує метод математичного сподівання з інтегруванням Повертає значення вибіркового середнього.

```
static double expectation(const Container& data);
```

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

7.3.2 Математичне сподівання квадрата

Обчислює середнє значення квадратів елементів емпіричної вибірки. Перевантажує метод другого початкового моменту неперервної випадкової величини. Повертає значення середнього квадрата.

```
static double second_moment(const Container& data);
```

$$\overline{x^2} = \frac{1}{n}\sum_{i=1}^{n} x_i^2$$

7.3.3 Дисперсія

Обчислює емпіричну дисперсію вибірки. Перевантажує дисперсію неперервної випадкової величини.

```
static double variance(const Container& data);
```

$$s^2 = \overline{x^2} - \bar{x}^2$$

Або еквівалентно: Обчислює емпіричну дисперсію вибірки за стабільною формулою. Ця реалізація є більш чисельно стабільною при великих значеннях.

```
static double variance_stable(const Container& data);
```

$$s^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

7.3.4 Виправлена дисперсія (Незміщена оцінка дисперсії.)

Обчислює виправлену дисперсію емпіричної вибірки.

```
static double variance_unbiased(const Container& data);
```

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

7.3.5 Середнє квадратичне відхилення

Обчислює емпіричне середнє квадратичне відхилення.

```
static double standard_deviation(const Container& data);
```

$$s = \sqrt{s^2}$$

7.3.6 Виправлене середнє квадратичне відхилення

```
static double standard_deviation_unbiased(const Container& data);
```

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

де

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

7.4 Виправлення

Дата	Причина	що виправлено
2026-03-21	Оновлення класу `DistributionStatistics`	Додано методи для емперичних даних: Математичне сподівання для вибірки, Математичне сподівання квадрата, Дисперсія, Виправлена дисперсія,
2026-03-21	Задача на оновлення класу `DistributionStatistics`	Додати окремими класами `ParameterEstimation` для методу максимальної правдоподібності та `StatisticalTests` для критерію Фішера.

`ParameterEstimation`

```
template<typename Container>
static double mle_mean(const Container& data);

template<typename Container>
static double mle_variance(const Container& data);

template<typename Container>
static double mle_stddev(const Container& data);
```

`StatisticalTests`

```
template<typename Container1, typename Container2>
static double fisher_statistic(const Container1& data1, const Container2& data2);
```

8 Щільність стандартного нормального розподілу

$$\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

$N(0,1)$:

```
double laplace_local_phi(const double x)
{
    return (1.0 / std::sqrt(2.0 * std::numbers::pi_v<double>)) * std::exp(-x * x / 2.0);
}
```

З параметрами a і σ, $N(a, \sigma^2)$:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-a)^2}{2\sigma^2}\right)$$

Запис через стандартну щільність:

$$f(x) = \frac{1}{\sigma}\varphi\left(\frac{x-a}{\sigma}\right)$$

```
double normal_density(double x, double a, double sigma)
{
    return (1.0 / sigma) * laplace_local_phi((x - a) / sigma);
}
```

9 Емпірична функція розподілу

10 ANOVA.

F критерій Фішера в методі дисперсійного аналізу ANOVA

10.1 Мета:

Перевірити, чи відрізняється середній дохід за добу між тарифами 1 день, 7 днів і 28 днів, та оцінити, чи тарифна політика відповідає очікуванню, що коротші проживання дають більший дохід за добу.

10.2 Постановка задачі

Розглядається сукупність проживань у хостелі, де існують три фіксовані тарифні групи:

Залежна змінна:

$$y = \text{revenuePerDay} = \frac{price}{\text{stayDays}}$$

Фактор:

$$g \in \{1Dday,\ 7Ddays,\ 28Days\}$$

10.3 Гіпотези

Нульова гіпотеза:

$$H_0 : \mu_1 = \mu_7 = \mu_{28}$$

Альтернативна гіпотеза:

$$H_1 : \text{принаймні одне середнє відрізняється}$$

10.4 Інтерпретація

Перевіряється, чи тарифна політика реально створює різницю у середньому доході за добу.

Очікування:

$$\mu_1 > \mu_7 > \mu_{28}$$

Якщо це виконується і статистично значуще, політика працює коректно.

Якщо ж:

$$\mu_7 \approx \mu_1 \quad \text{або} \quad \mu_{28} \geq \mu_7$$

це сигнал про неефективність або особливості даних.

10.5 Підготовка даних

Обчислення тривалості:

$$stayDays = dateOut - dateIn$$

Фільтрація:

$$stayDays = 1 \quad \text{або} \quad 7 \quad \text{або} \quad 28$$

10.6 Формування вибірок

- Група 1: всі значення $revenuePerDay$ при $stayDays = 1$
- Група 2: всі значення $revenuePerDay$ при $stayDays = 7$
- Група 3: всі значення $revenuePerDay$ при $stayDays = 28$

10.7 Розрахунок ANOVA

- Кількість груп: $k = 3$
- Обсяг вибірки: $N = n_1 + n_7 + n_{28}$
- Середні по групах: $\bar{x}_1, \bar{x}_7, \bar{x}_{28}$

Загальне середнє $\bar{x}$:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{k} \sum_{j=1}^{n_i} x_{ij}$$

або

$$\bar{x} = \frac{n_1 \bar{x}_1 + n_7 \bar{x}_7 + n_{28} \bar{x}_{28}}{N}$$

Ступені свободи:

$df_{between} = k - 1$ означає, що хоча є k груп, їх середні не повністю незалежні, бо вони пов'язані через загальне середнє, тому вільних лише $k - 1$.

$df_{within} = N - k$ з усіх N спостережень вже "витратили" по одному параметру на кожну групу, тобто k середніх, і залишилось $N - k$ незалежних відхилень всередині груп.

Міжгрупова сума квадратів:

$$SS_{between} = \sum_{i=1}^{k} n_i(\bar{x}_i - \bar{x})^2$$

Внутрішньогрупова сума квадратів:

$$SS_{within} = \sum_{i=1}^{k} \sum_{j=1}^{n_i} (x_{ij} - \bar{x}_i)^2$$

Середні квадрати:

- зважена об'єднана дисперсія між групами (масштабована міжгрупова варіація):

$$MS_{between} = \frac{SS_{between}}{df_{between}}$$

- зважена об'єднана дисперсія всередині всіх груп (сумарна внутрішня варіація):

$$MS_{within} = \frac{SS_{within}}{df_{within}}$$

10.8 Критерій Фішера

$$F = \frac{MS_{between}}{MS_{within}}$$

10.9 Критичне значення через квантиль F розподілу

$$F_{\text{crit}} = F^{-1}(1-\alpha;\ df_{between},\ df_{within})$$

Де α - рівень значущості, $df_{between}$ - ступенів свободи між групами, df_{within} - ступенів свободи всередині груп.

α	Інтерпретація
0.05	стандартний рівень значущості
0.01	нижча допустима ймовірність помилки першого роду
0.001	дуже низька ймовірність помилки першого роду

10.10 Висновок

```
=== PERIOD: 2019-01-01 - 2019-12-31 ===

Статистика:
   group        n         mean       var        min        max       median      p25        p75
==========================================================================================
     1        2873      168.76     519.08     120.00     250.00     170.00     150.00     180.00
     7        2384      104.47     428.98      78.57     271.43     100.00      92.86     107.14
    28        1374      70.36      95.75      58.93     160.71      67.86      66.07      71.43

F critical: 2.99709
ANOVA F: 13294.4

=== PERIOD: 2020-01-01 - 2020-12-31 ===

Статистика:
   group        n         mean       var        min        max       median      p25        p75
==========================================================================================
     1        1409      188.74     495.85     150.00     250.00     180.00     180.00     200.00
     7        1862      113.92     485.48      85.71     200.00     107.14     100.00     128.57
    28        1183      73.73      69.47      64.29     160.71      71.43      71.43      75.00

F critical: 2.99775
ANOVA F: 11957.3

=== PERIOD: 2021-01-01 - 2021-12-31 ===

Статистика:
   group        n         mean       var        min        max       median      p25        p75
==========================================================================================
     1        1988      184.06     670.44     150.00     250.00     180.00     160.00     200.00
     7        2322      113.48     540.69      85.71     250.00     107.14     100.00     128.57
    28        1464      76.92      200.55      64.29     160.71      71.43      71.43      78.57

F critical: 2.99729
ANOVA F: 10574.4
```

Рис. 18 Вимірювання по інтервалах

Значення F порівнюється з критичним значенням F-розподілу:

$$F_{\text{crit}} \approx 2.997 \quad \text{при } \alpha = 0.05$$

У всіх розглянутих періодах виконується:

$$F_{\text{obs}} \gg F_{\text{crit}}$$

Отже, нульова гіпотеза

$$H_0 : \mu_1 = \mu_7 = \mu_{28}$$

відкидається. Різниця між тарифними групами є статистично значущою.

Якщо різниця статистично значуща, робиться висновок, що дохід за добу залежить від тривалості проживання.

10.10.1 1. Перевірка гіпотези

У всіх трьох періодах спостерігається чіткий порядок середніх:

2019	$\mu_1 = 168.76,\ \mu_7 = 104.47,\ \mu_{28} = 70.36$
2020	$\mu_1 = 188.74,\ \mu_7 = 113.92,\ \mu_{28} = 73.73$
2021	$\mu_1 = 184.06,\ \mu_7 = 113.48,\ \mu_{28} = 76.92$

У всіх випадках виконується:

$$\mu_1 > \mu_7 > \mu_{28}$$

Це прямо відповідає очікуванню тарифної політики.

10.10.2 2. Стійкість по роках

Структура не змінюється:

- порядок середніх стабільний
- медіани підтверджують ті ж самі значення
- аквартилі не перекриваються критично між групами

Це означає, що ефект не разовий, а стабільний у часі.

F дуже велике у всіх роках:

2019	2020	2021
13294	11957	10574

Міжгрупова варіація значно більша за внутрішньогрупову. Тобто різниця між тарифами не випадкова, а системна. Нульова гіпотеза $H_0 : \mu_1 = \mu_7 = \mu_{28}$ відкидається з дуже високою впевненістю. Короткі проживання дають більший дохід за добу. Це означає, що знижка за довгий термін реально працює, клієнт “платить менше за день”, але довше днів живе.

10.11 Реалізація:

Демонстраційна реалізація зроблена на прикладі ненормалізованих даних, мала вирізка яких показано у файлі example_non_normalized_sample_csv.txt. Файл містив близько 27000 рядків. Методами класу `CsvToHostelStayMapper`, дані були зчитані, нормалізовані, а потім методами класу `HostelStayRepository` завантажені в таблицю БД, міграція якої описана у файлі 001_accommodation.sql. Покроково виводилися за допомогою `SQL`-запитів етапи обчислення, які описані у файлі FISHER_PLPGSQL_FUNCTION_OUTPUT.md, і потім на їх основі були побудовані дві функції мовою PL/pgSQL. Функція `stats_by_group` повертає структуру зі статистичними обчисленнями, а

функція `calculate_fisher` повертає значення критерія Фішера. Обидві функції приймають параметрами початок інтервалу та кінець. Якщо параметри не задані, обчислення виконується для всього періоду.

- `calculate_fisher` повертає одне скалярне значення:

```
PGresult* res = PQprepare(
    conn,
    "calculate_fisher_stmt",
    R"sql(
        SELECT calculate_fisher($1, $2)
    )sql",
    2,
    nullptr
);
```

Обидві функції знаходяться у другому файлі міграції 002_anova_functions.sql. Таким чином, при першому запуску контейнера, разом із додаванням схеми, також додаються і функції в метадані `PostgreSQL`. Після цього вони доступні для виклику з коду `C++` як звичайні агрегаційні функції `sql`.

- `stats_by_group` повертає таблицю:

```
PGresult* res = PQprepare(
    conn,
    "stats_by_group_stmt",
    R"sql(
        SELECT *
        FROM stats_by_group($1, $2)
    )sql",
    2,
    nullptr
);
```

Деякі виводи безпосередньо з БД під час налагодження:

```
-- Зведена таблиця значущіх даних
SELECT
    stay_days AS "група",
    COUNT(*) AS "кількість_в_групі",
    AVG(revenue_per_day) AS "середнє_по_групі",
    VARIANCE(revenue_per_day) AS "дисперсія_всередині_групи",
    MIN(revenue_per_day) AS "мінімум",
    MAX(revenue_per_day) AS "максимум",
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY revenue_per_day) AS "медіана",
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY revenue_per_day) AS "p25",
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY revenue_per_day) AS "p75"
```

Output | Зведена таблиця значущіх даних

	група	кількість…	середнє_по_групі	дисперсія_всередині_групи	мінімум	максимум	медіана	p25	p75
1	1	9519	178.1426620443323879	772.9525388678080077	100	250	180	160	200
2	7	9871	110.4485999392158849	472.3783885158216782	71.43	271.43	107.14	100	114.29
3	28	6376	75.3192597239648683	139.6913329421015081	55.36	160.71	75	67.86	78.57

Рис. 19 Зведена статистика по тарифах

```
-- зважена об'єднана дисперсія між групами (масштабована міжгрупова варіація)
WITH filtered_data AS (
    SELECT
        stay_days,
        revenue_per_day
    FROM hostel_stays
    WHERE stay_days IN (1, 7, 28)
),
групи AS (
```

Output | зважена об'єднана ди... міжгрупова варіація)

	ms_between
1	22315000.0463125650895278

Рис. 20 зважена об'єднана дисперсія між групами (масштабована міжгрупова варіація)

```
-- зважена об’єднана дисперсія всередині всіх груп (сумарна внутрішня варіація)
WITH filtered_data AS (
    SELECT
        stay_days,
        revenue_per_day
    FROM hostel_stays
    WHERE stay_days IN (1, 7, 28)
),
```

Output | зважена об’єднана ди...а внутрішня варіація)

	ms_within
1	501.1011608547472614

Рис. 21 зважена об’єднана дисперсія всередині всіх груп (сумарна внутрішня варіація)

```sql
-- pivot таблиця
SELECT
    EXTRACT(MONTH FROM checkin_date) AS місяць,
    AVG(revenue_per_day) FILTER (WHERE EXTRACT(YEAR FROM checkin_date) = 2018) AS "2018",
    AVG(revenue_per_day) FILTER (WHERE EXTRACT(YEAR FROM checkin_date) = 2019) AS "2019",
    AVG(revenue_per_day) FILTER (WHERE EXTRACT(YEAR FROM checkin_date) = 2020) AS "2020",
    AVG(revenue_per_day) FILTER (WHERE EXTRACT(YEAR FROM checkin_date) = 2021) AS "2021",
```

Output | pivot таблиця

CSV

	місяць	"2018"	"2019"	"2020"	"2021"	"2022"	"2023"	"2024"
1	1	130	112.1757610789980732	132.2698275862068966	121.1025722543352601	129.9172841726618705	125.9454320987654321	130.407190332326284
2	2	<null>	110.9531115459882583	124.2204555808656036	124.8027894736842105	127.5459725400457666	129.3796137339055794	134.0619685039370079
3	3	<null>	106.6138146551724138	126.9271578947368421	130.3522781774580336	105.4816279069767442	126.6901532567049808	147.9672
4	4	<null>	109.7600225225225225	115.9806511627906977	126.9002583979328165	115.486969696969697	130.8526808510638298	<null>
5	5	<null>	115.3511378205128205	123.701491935483871	131.3663021868787276	116.6508421052631579	135.0536092715231788	<null>
6	6	<null>	123.6246620450606586	128.7764067796610169	127.3349105367793241	127.591271676300578	139.3449828178694158	<null>
7	7	<null>	128.2582978723404255	127.8326888217522659	126.5029158110882957	122.905	140.0124923076923077	<null>
8	8	<null>	126.8213306451612903	130.2841095890410959	132.9724077328646749	132.1565876777251185	134.7771387283236994	<null>
9	9	<null>	131.0859550561797753	126.6259600997506234	132.6471669477234401	137.1148207171314741	145.9365465465465465	<null>
10	10	104.8498335854765507	138.9649450549450549	129.2022631578947368	126.2752886405959032	139.3765263157894737	133.5819242902208202	<null>
11	11	99.608655737704918	144.2169616026711185	121.0532920792079208	125.7604545454545455	123.1563346613545817	130.2501226993865031	<null>
12	12	111.572570281124498	143.0392520325203252	128.913715710723192	131.4066483516483516	130.7553207547169811	130.6875987841945289	<null>

Рис. 22 pivot таблиця середніх значень по роках

Docker:

- Запуск контейнера

```
docker compose up -d
```

- Зупинка та повне очищення середовища

```
docker compose down -v
```

- Примусово видаляє контейнер бази даних

```
docker rm -f nova-db
```

- Перегляд усіх `volume`

```
docker volume ls
```

- Детальна інформація про конкретний `volume`

```
docker volume inspect 2026-03-26-anova_data
```

- Перегляд використання `volume` контейнерами

```
docker ps -a --filter volume=2026-03-26-anova_data
```

- Ручне видалення `volume`

```
docker volume rm 2026-03-26-anova_data
```

---

11 Тема 6. Інтервальні оцінки параметрів

11.1 Мета

Освоєння побудови довірчих інтервалів для математичного сподівання нормального розподілу при відомому середньому квадратичному відхиленні, аналіз впливу обсягу вибірки та дисперсії на ширину інтервалу.

11.2 Завдання

Для кожного варіанта, заданого вибірковим середнім $\bar{x}$, обсягом вибірки n та відомим стандартним відхиленням σ, побудувати надійний (довірчий) інтервал для оцінювання математичного сподівання нормального розподілу з надійністю (рівнем довіри) $\gamma = 0.95$.

- середнє `x_bar`

- обсяг вибірки `n`
- стандартне відхилення `sigma`

```
struct task
{
    std::string id;
    double x_bar;
    int n;
    double sigma;
};

std::vector<task> tasks =
{
    {"1", 75.17, 36, 6},
    {"2", 75.16, 49, 7},
    {"3", 75.15, 64, 8},
    {"4", 75.14, 81, 9},
    {"5", 75.13, 100, 10},
    {"6", 75.12, 121, 11},
    {"7", 75.11, 144, 12},
    {"8", 75.10, 169, 13},
    {"9", 75.09, 196, 14},
    {"10", 75.08, 225, 15}
};
```

11.3 Розв’язання для Задачі №1:

З рівняння:

$$2\Phi(t) = 0.95 \Rightarrow \Phi(t) = 0.475$$

Де $\Phi(z)$ - функція Лапласа (табульована), що визначає ймовірність на інтервалі від 0 до z.

Нам потрібен інтервал

$$P(\bar{x} - \varepsilon < a < \bar{x} + \varepsilon) = 2\Phi(t) = 0.475 + 0.475 \approx 0.95$$

ймовірність поза інтервалом $1 - 0.95 = 0.05$

Оскільки інтервал симетричний $0.05/2$

- зліва 0.025
- справа 0.025

З таблиці функції Лапласа знаходимо таке значення t, для якого

$$\Phi(t) = 0.475$$

За таблицею функції Лапласа:

$$t \approx 1.96$$

Еквівалентно, це означає, що t є квантилем стандартного нормального розподілу рівня 0.975:

$$t_{0.975} \approx 1.96$$

Похибка:

$$\varepsilon = \frac{t\sigma}{\sqrt{n}} = \frac{1.96 \cdot 6}{\sqrt{36}} = \frac{1.96 \cdot 6}{6} = 1.96$$

Довірчий інтервал:

$$h_1 = \bar{x} - \varepsilon = 75.17 - 1.96 = 73.21$$

$$h_2 = \bar{x} + \varepsilon = 75.17 + 1.96 = 77.13$$

12 Реалізація обчислення

1. Перехід від γ до центральної ймовірності

```
double central_probability(double gamma)
{
    return (1.0 + gamma) / 2.0;
}
```

2. Квантиль стандартного нормального розподілу

```
#include <boost/math/distributions/normal.hpp>

double normal_quantile(double p)
{
    boost::math::normal_distribution<> dist(0.0, 1.0);
    return boost::math::quantile(dist, p);
}
```

3. Похибка ε

```
double margin_of_error(double t, double sigma, int n)
{
    return t * sigma / std::sqrt(n);
}
```

4. Головна функція інтервалу

```
std::pair<double, double> confidence_interval(
    double x_bar,
    int n,
    double sigma,
    double gamma
)
{
    const double p = central_probability(gamma);
    const double t = normal_quantile(p);
    const double eps = margin_of_error(t, sigma, n);

    return {x_bar - eps, x_bar + eps};
}
```

5. Функція що застосовує `confidence_interval` до кожного завдання `task` і повертає вектор результатів:

```
struct result
{
    std::string id;
    double h_1;
    double h_2;
};

std::vector<result> compute_intervals(
```

```
    const std::vector<task>& tasks,
    double gamma
)
{
    std::vector<result> results;
    results.reserve(tasks.size());

    std::for_each(tasks.begin(), tasks.end(), [&](const task& t)
    {
        const auto interval = confidence_interval(t.x_bar, t.n, t.sigma, gamma);

        results.push_back(result{
            t.id,
            interval.first,
            interval.second
        });
    });

    return results;
}
```

12.1 Аналіз

```
Task 1
x_bar =  75.17   n =  36   sigma = 6.00   gamma = 0.95
Interval   h_1 =  73.21   h_2 =  77.13

Task 2
x_bar =  75.16   n =  49   sigma = 7.00   gamma = 0.95
```

```
Interval   h_1 =  73.20   h_2 =  77.12

Task 3
x_bar =  75.15   n =  64   sigma = 8.00   gamma = 0.95
Interval   h_1 =  73.19   h_2 =  77.11

Task 4
x_bar =  75.14   n =  81   sigma = 9.00   gamma = 0.95
Interval   h_1 =  73.18   h_2 =  77.10

Task 5
x_bar =  75.13   n = 100   sigma = 10.00   gamma = 0.95
Interval   h_1 =  73.17   h_2 =  77.09

Task 6
x_bar =  75.12   n = 121   sigma = 11.00   gamma = 0.95
Interval   h_1 =  73.16   h_2 =  77.08

Task 7
x_bar =  75.11   n = 144   sigma = 12.00   gamma = 0.95
Interval   h_1 =  73.15   h_2 =  77.07

Task 8
x_bar =  75.10   n = 169   sigma = 13.00   gamma = 0.95
Interval   h_1 =  73.14   h_2 =  77.06

Task 9
x_bar =  75.09   n = 196   sigma = 14.00   gamma = 0.95
Interval   h_1 =  73.13   h_2 =  77.05
```

```
Task 10
x_bar =  75.08   n = 225   sigma = 15.00   gamma = 0.95
Interval   h_1 =  73.12   h_2 =  77.04
```

- дисперсія $\sigma^2 = D(X)$
- похибка $\varepsilon = \frac{t\sqrt{D(X)}}{\sqrt{n}} = \frac{t\sigma}{\sqrt{n}}$

Якщо зростає дисперсія, то зростає σ, а отже і похибка ε, тому інтервал розширюється.

Ширина інтервалу:

$$L = 2\varepsilon = 2\frac{t\sigma}{\sqrt{n}}$$

Отже, L прямо пропорційна до σ і обернено пропорційна до $\sqrt{n}$.

12.1.1 Закономірності по завданнях

- $n = 36, 49, 64, \dots, 225 = k^2$
- $\sqrt{n} = 6, 7, 8, \dots, 15$
- $\sigma = 6, 7, 8, \dots, 15$

Тому:

$$\frac{\sigma}{\sqrt{n}} = 1$$

$$\varepsilon = t \cdot \frac{\sigma}{\sqrt{n}} = t = 1.96$$

$$L = 2\varepsilon = 3.92$$

12.2 Висновок

Зі зростанням σ інтервал має розширюватися, а зі зростанням n звужуватися. Сукупність усіх завдань не показова, бо ефекти компенсуються, оскільки σ і $\sqrt{n}$ зростають однаково. Тому ширина довірчого інтервалу не змінюється.